Table of Contents

COVER PAGE

Project Title: TalkToJesus — A Bilingual Voice-First AI Spiritual Companion

Student Name & Roll Number: Manish Rachakonda (2023EBCS668)

Program: BSc Computer Science (Online Mode)

Institution: Birla Institute of Technology and Science, Pilani (BITS Pilani)

Academic Year: 2025–2026

Internal Supervisor: Swapnil Saurav

Contact: 2023ebcs668@online.bits-pilani.ac.in

Source code: https://github.com/manish-gitx/rn-final-manish

Date: **25 August 2026**

DECLARATION

I hereby declare that this capstone project titled “TalkToJesus — A Bilingual Voice-First AI Spiritual Companion” is an original work carried out by me and has not been submitted to any other university or institution for the award of any degree or diploma. All external libraries, services, media assets and reference material used in the project are acknowledged in the References section and in Appendix E, and all third-party licence obligations are recorded there.

Where this report restates conclusions from the Study Project Phase 1, Phase 2 and Phase 3 documents, those documents are cited as prior work of my own. Section 1.6 records every place where the implementation departed from that earlier design, and Chapter 3 records every place where a measurement contradicts a figure quoted in those earlier documents.

Name: **Manish Rachakonda**

Roll Number: **2023EBCS668**

Signature: **Manish Rachakonda** Date: **25 August 2026**

Supervisor's approval

Certified that the work described in this report was carried out by the candidate under my supervision.

Supervisor: Swapnil Saurav

Signature: **Swapnil Saurav** Date: **25 August 2026**

ABSTRACT

Bible applications in Telugu have been downloaded by millions of users, yet they present static content — scripture, devotionals and recorded songs — and offer no way to ask a question and receive an answer. General-purpose conversational assistants can hold a dialogue but are neither grounded in scripture nor available in Telugu with culturally appropriate forms of address. This project delivers TalkToJesus, a mobile application in which a user speaks aloud in English or Telugu and receives a spoken, scripture-grounded reply in a first-person pastoral voice.

The system consists of a Flutter mobile application for iOS and Android, a 28-endpoint Express and TypeScript API deployed to Google Cloud Run, a PostgreSQL database provisioned through Supabase across eight tables, and a dependency-free administrative console served by the same API. A conversational turn is a three-stage pipeline: OpenAI Whisper transcribes the recording, OpenAI GPT-4o generates a reply under a language-specific persona prompt with the previous turns supplied as context, and ElevenLabs synthesises that reply as speech. Subscriptions are handled through Razorpay with HMAC-SHA256 webhook verification, and five runtime feature flags allow the free-tier limit, maintenance mode, speech synthesis and the context window to be changed without redeployment.

The engineering contribution that distinguishes this system from a wrapper around a language model is that every turn is instrumented. Each conversation record stores the duration of transcription, generation and synthesis separately alongside the total, so the cost of each stage is a measured quantity rather than an assumption. Because the text-input path skips

transcription and records a zero, the price of speech recognition is directly observable in the same table.

Validation used 141 automated tests — 66 backend cases across 10 Jest suites and 75 Flutter cases across 9 files — all passing, together with a functional walkthrough of the complete subscription and conversation flow on physical hardware. The instrumentation produced the project's most consequential result and also contradicted its own earlier documentation: against a non-functional requirement of five seconds, the measured end-to-end latency on the live instance was 27,457 ms at the median, of which speech synthesis alone accounted for 19,618 ms. The three-to-six second figure quoted in the Phase 3 document and in the presentation deck is traceable to the administrative console's demonstration fixtures and is not a measurement. That requirement is therefore recorded as not met, and Section 3.5 sets out what the measurement implies for the design.

Keywords: voice interface, speech recognition, speech synthesis, large language models, bilingual interfaces, Telugu, Flutter, latency instrumentation

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Term	Expansion
AAC	Advanced Audio Coding
API	Application Programming Interface
CD	Continuous Deployment
CI	Continuous Integration
CVD	Colour Vision Deficiency
FR	Functional Requirement
HMAC	Hash-based Message Authentication Code
JWT	JSON Web Token
LLM	Large Language Model

Term	Expansion
MRR	Monthly Recurring Revenue
NFR	Non-Functional Requirement
OAuth	Open Authorization
p50 / p95	50th / 95th percentile
RLS	Row Level Security
REST	Representational State Transfer
SDK	Software Development Kit
STT	Speech To Text
TTS	Text To Speech
UVP	Unique Value Proposition

CHAPTER 1: INTRODUCTION

The problem, the requirements and the system design were settled across the three Study Project phases. What follows is a condensed restatement of that work. Where building the system forced a departure from it, Section 1.6 sets out the departure and why it happened.

1.1 Overview of the project

TalkToJesus is a mobile application that lets a believer speak a question aloud and receive a spoken reply grounded in scripture. The user holds the phone, presses a single control, says what is on their mind in English or Telugu, and hears a response in a synthesised voice a few moments later. The reply is written in the first person, addresses the user as “My child” in English or “నా బిడ్డ” in Telugu, cites scripture by book, chapter and verse, and closes with a short prayer.

Around that central interaction the application provides a bilingual Bible reader with six translations including two in Telugu, a library of twelve public-domain hymns, a transcript of the user’s previous conversations, and a subscription that lifts a three-conversation free-tier limit.

The system is not a single application. It comprises a Flutter client for iOS and Android, a TypeScript REST API running on Google Cloud Run, a PostgreSQL database provisioned

through Supabase, an administrative console served as static files by the same API, and integrations with four external services: Google for identity, OpenAI for transcription and generation, ElevenLabs for speech synthesis, and Razorpay for payments.

1.2 Problem statement and motivation

Telugu-language Bible applications have accumulated millions of downloads. They are well made and widely used, and they share one characteristic: the content flows in one direction. A user can read a passage, follow a reading plan or play a recorded song, but cannot ask what a passage means for a decision they are facing on a particular evening.

Four gaps follow from that.

Communication is one-way. Existing applications present content. None accept a question and produce an answer specific to it.

Access to pastoral conversation is limited. Speaking to a pastor or elder requires that another person be available, that the conversation be scheduled, and that the person seeking guidance be willing to raise the subject with someone they know. For a question asked at two in the morning, or one the user is embarrassed to voice aloud to an acquaintance, none of those conditions hold.

Guidance is not context-aware. A reading plan does not know what the reader is worried about. A search returns passages containing a keyword, not passages that address a situation.

Language and culture are a barrier. General-purpose conversational assistants can hold a dialogue, but they are not grounded in scripture, they do not adopt a pastoral register, and their Telugu is not idiomatic in a devotional context. Addressing a believer correctly in Telugu is not a translation problem; "నా బిడ్డ" carries a weight that a literal rendering of "my child" does not.

The motivation for the project is the conjunction of these four. Each is individually addressed somewhere in the market. None of the existing systems addresses all four at once, and the combination is what a Telugu-speaking believer with a question at an inconvenient hour actually needs.

1.3 Objectives of the capstone

The Study Project defined seven primary and four secondary objectives. They are restated here because Chapter 6 assesses the implementation against them.

Primary objectives

PO1. Build a cross-platform mobile application providing AI-driven spiritual conversation grounded in biblical teaching.

PO2. Build a backend API with secure authentication, conversation management and subscription handling.

PO3. Integrate a large language model to produce context-aware, scripture-based responses that adapt to the selected language.

PO4. Provide voice interaction through speech-to-text input and text-to-speech output.

PO5. Support bilingual operation in English and Telugu with dynamic switching and culturally appropriate responses.

PO6. Integrate a payment gateway for subscription monetisation with a free tier and premium plans.

PO7. Deploy the backend on Google Cloud Platform using containerisation for automatic scaling.

Secondary objectives

SO1. Instrument the application with product analytics and error tracking.

SO2. Provide a curated library of spiritual music.

SO3. Support offline use through local caching and connectivity monitoring.

SO4. Provide Bible reading with book, chapter and verse navigation.

1.4 Scope of implementation

In scope, and delivered. Google sign-in; the voice conversation pipeline in both languages; the bilingual Bible reader with six translations and an offline cache; the bundled hymn library and player; Razorpay subscriptions with webhook reconciliation; the conversation transcript;

the administrative console with live metrics, content management, webhook audit, runtime feature flags and an audit trail; deployment to Cloud Run with a continuous integration pipeline.

In scope, delivered differently than designed. Multi-turn context was designed as a Capstone extension and was implemented; Section 1.6 records this. Text input to the conversation exists as a working API endpoint but has no user interface in the shipped build.

Out of scope. Publication to the App Store and Google Play. A video or animated avatar. Any form of community, sharing or social feature. Human pastoral escalation. Retrieval-augmented generation over a scripture corpus — the system supplies context by replaying previous conversational turns, not by retrieving passages from an indexed corpus, and Section 2.4.4 states this precisely because the distinction is easy to overstate.

1.5 Organization of the report

The remainder of this report is arranged as follows. Chapter 2 sets out how the system is built — its architecture, the technologies chosen and why, the individual modules, and the handful of algorithms that carry the real complexity. Chapter 3 is the evidence chapter: how the system was tested, what the measurements showed, which defects were found, and — equally important — which properties were never verified at all. Chapter 4 deals with running and deploying the system. Chapter 5 collects the record of how the work was actually carried out. Chapter 6 draws conclusions and sets out what should be done next. Five appendices follow: a user manual, an installation guide, a source code reference, the demonstration video links, and the third-party licensing record.

1.6 Changes from the Study Project design

Implementation departed from the Phase 2 and Phase 3 documents in the following ways. These are recorded here rather than left implicit, because several of them contradict statements in documents that have already been submitted.

The mobile framework is Flutter, not React Native. The repository is named `rn-final-manish`, which invites the opposite conclusion. It contains no React Native code. The client is Flutter 3.38.6 with Dart, using Riverpod for state management. The repository name is a historical artefact of an early decision that was reversed before any code was written.

Multi-turn context was implemented, not deferred. Phase 3 lists both conversation history and multi-turn context as Capstone future work. Both shipped. The conversation controller loads the previous six turns from the database and replays them into the model's message array; the window size and whether the feature is active at all are runtime flags.

An administrative console was added. No phase document mentions it. It comprises eighteen API routes and a single-file web console with six tabs, and it is the source of every measurement in Chapter 3.

Runtime feature flags were added. Five flags — the free-tier limit, maintenance mode, speech synthesis, multi-turn context and the context window size — are stored in the database and read on every conversational turn behind a fifteen-second cache. None of this was in the Phase 2 design.

A webhook audit trail was added. Every Razorpay webhook is recorded with the result of its signature check, including the ones that fail. Recording failures deliberately preserves the evidence that verification runs at all.

Continuous integration was added. A GitHub Actions workflow builds and tests both tiers and verifies the container image on every push to `main`.

The measured pipeline latency is an order of magnitude worse than reported. Phase 3 and the presentation deck state a three-to-six second pipeline. The live instance measures 27,457 ms at the median. Section 3.5 establishes where the three-to-six second figure came from and why it is not a measurement. This is the single most important correction in this report.

The automated test counts in Phase 3 are wrong. Phase 3 reports 40 backend tests with two failing and 65 frontend tests. The current figures are 66 backend and 75 frontend, none failing. The two failures were fixed, and Phase 3's stated root cause for them was itself incorrect; Section 3.6 gives the actual cause.

Cold start is 4.4 seconds, not two. Phase 3 estimates roughly two seconds for a Cloud Run cold start. The measured figure recorded in the demonstration script is 4.4 seconds cold against approximately 0.4 seconds warm.

CHAPTER 2: IMPLEMENTATION DETAILS

2.1 System architecture and design

2.1.1 High-level architecture

The system is a conventional three-tier deployment. The distribution of responsibility across the tiers is worth stating explicitly, because one decision in the data tier propagates through everything above it.

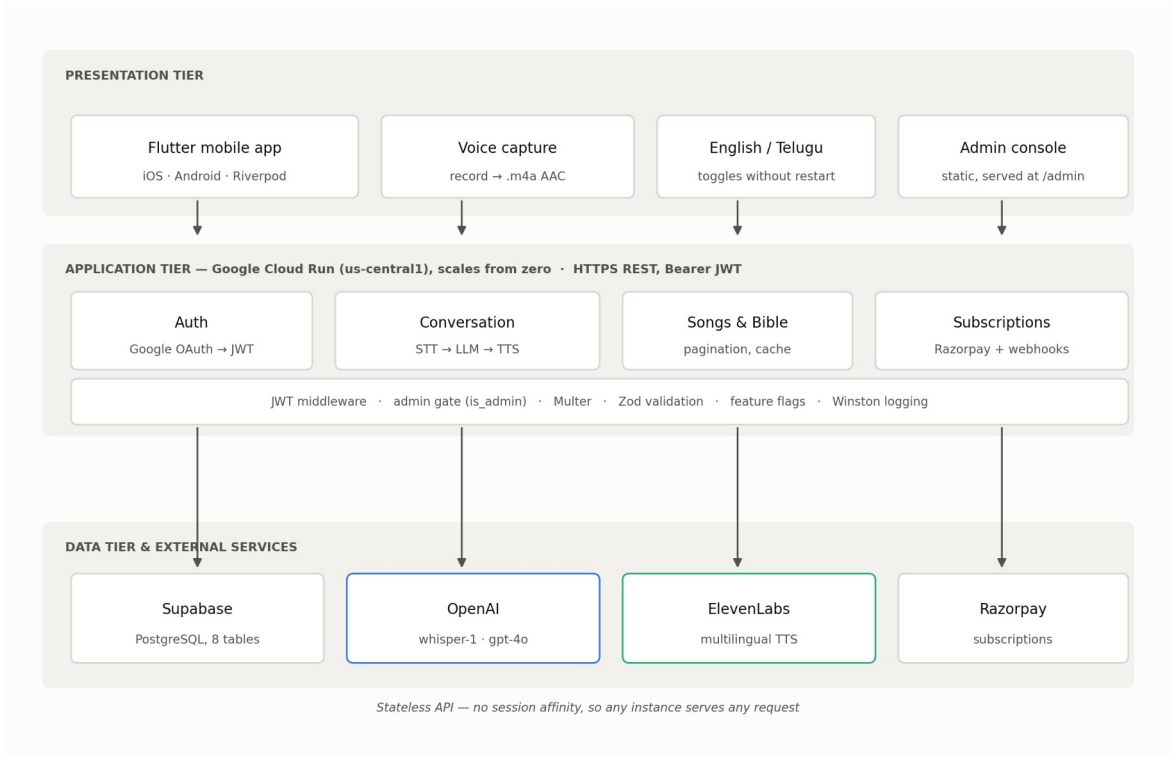

Figure 26 — Three-tier system architecture. The Flutter client and the administrative console both speak to a single stateless API on Cloud Run, which in turn talks to Supabase and to four external services.

The **presentation tier** is the Flutter application, plus the administrative console, which is a single 995-line HTML file with no build step and no external dependencies. Both authenticate with a bearer token and speak the same REST API.

The **application tier** is a stateless Express 5 service on Node 20, deployed to Cloud Run in `us-central1` and configured to scale from zero. Statelessness is not incidental: because no session state is held in process memory, any instance can serve any request, which is what permits scale-to-zero without a shared cache.

The **data tier** is Supabase-hosted PostgreSQL across eight tables, together with OpenAI, ElevenLabs and Razorpay.

The consequential decision is that the API authenticates to PostgreSQL with a service-tier key. Row-level security is enabled on every table, but the service-tier key bypasses it by design. **Authorization is therefore enforced entirely in application code**, not by the database. Every endpoint that returns user-scoped data must filter by the authenticated user's identifier itself, because the database will not do it. Section 2.4.5 examines the consequence.

2.1.2 Data flow — speech to spoken reply

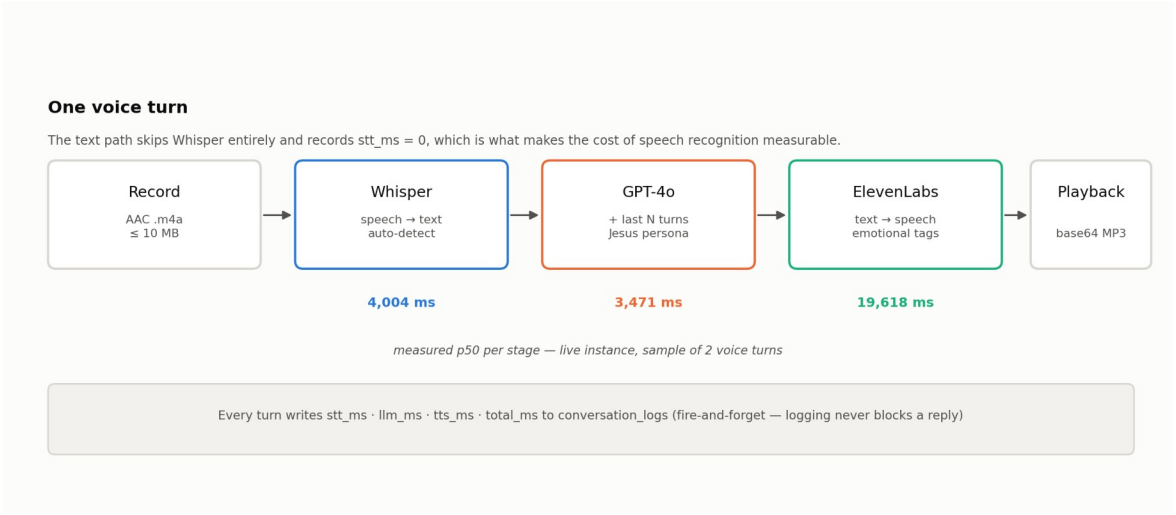

Figure 27 — One voice turn, end to end, annotated with the measured per-stage timings.

A voice turn proceeds as follows.

1. The user presses the voice bar. The client checks microphone permission and handles four distinct outcomes — granted, denied, permanently denied, restricted — with a specific dialog for each; the permanently-denied case offers to open system settings.
2. Recording begins. The client captures AAC-LC at 128 kbps, 44.1 kHz, mono, into an `.m4a` file in the temporary directory. The user sees a pulsing indicator with a cancel control and a stop control.
3. On stop, the file is uploaded as `multipart/form-data` to `POST /api/conversation/send-message` along with a language field. Uploads are capped at 10 MB and restricted by MIME type.

4. The API authenticates the bearer token, then checks entitlement: the turn proceeds if the user's conversation count is below the free-tier limit or if a subscription is active. Otherwise the request is refused with HTTP 402.
5. Whisper transcribes the audio. The language parameter is deliberately **not** passed to Whisper, which auto-detects instead; Section 2.4.2 explains why.
6. The previous turns are loaded from the database and folded into the message array, and GPT-4o generates a reply under the language-specific persona prompt.
7. ElevenLabs synthesises the reply. The audio is returned as a base64 data URI.
8. The client decodes the audio to a temporary file, plays it, displays the text, and deletes the temporary file.

Each of steps 5, 6 and 7 is timed independently, and all four durations are written to the database after the response has been returned to the client.

2.1.3 Component interaction — authentication and entitlement

Two gates stand in front of a conversational turn, and they answer different questions.

Authentication answers "who is this?". The client obtains a Google ID token and posts it to `/api/auth/create-or-get-user`. The server verifies the token against three OAuth client identifiers in sequence — web, iOS and Android — because a single application distributed on two platforms plus a web console legitimately presents tokens minted for three different audiences. On success the user row is created or its last-login timestamp updated, and the server mints its own HS256 JWT with a seventy-day expiry.

Entitlement answers "may this user do this?". It is evaluated per turn, not per session, because a user's free-tier allowance is consumed during a session.

A third gate applies only to administrative routes. It reads `is_admin` from the user record already loaded by the authentication middleware, requires the value to be strictly `true`, and — when it fails — returns **404, not 403**. Returning "not found" rather than "forbidden" avoids confirming to an unauthenticated prober that the administrative surface exists at that path.

2.2 Technology stack

Table 3 — Technology stack and rationale

Layer	Technology	Version	Rationale
Mobile framework	Flutter / Dart	3.38.6 / 3.9	One codebase for iOS and Android; the audio recording and playback packages are mature
Mobile state	Riverpod	2.5	Compile-time safe providers; no `BuildContext` dependency for business logic
Mobile local store	sqflite	2.4	Offline Bible cache with a least-recently-used eviction policy
Mobile HTTP	package:http	1.5	A single hand-written client was simpler than configuring an interceptor stack for ten endpoints
Backend runtime	Node.js	20 LTS	Long-term support; matches the Cloud Run base image
Backend framework	Express	5.1	Minimal, well understood, adequate for 28 routes
Backend language	TypeScript	5.9	`strict` mode; compilation doubles as a typecheck gate in CI
Validation	Zod	4.1	Schema validation that also produces

Layer	Technology	Version	Rationale
			the TypeScript type
Database	Supabase (PostgreSQL)	—	Managed Postgres with an SDK; no ORM was introduced
Identity	Google OAuth 2.0 + JWT	—	Users already have a Google account; no password storage
Speech to text	OpenAI Whisper	`whisper-1`	Strong Telugu recognition; automatic language detection
Generation	OpenAI GPT-4o	`gpt-4o`	Instruction-following adequate to hold a persona across a long system prompt
Speech synthesis	ElevenLabs	multilingual	Telugu voice quality; supports inline emotional direction
Payments	Razorpay	2.9	Subscription primitives and UPI support for the Indian market
Container platform	Google Cloud Run	—	Scales to zero; the workload is bursty and mostly idle
Build & deploy	Docker + Cloud Build	—	Multi-stage image; deploys on push to `main`

Layer	Technology	Version	Rationale
Continuous integration	GitHub Actions	—	Builds and tests both tiers plus the container image
Error tracking	Sentry	9.6	Crash and error capture with session replay
Product analytics	PostHog	5.5	Event tracking
Logging	Winston + Morgan	3.18	Structured logs with per-environment levels

2.3 System modules

2.3.1 Authentication module

Three source files carry this module: the auth service, the JWT utility and the authentication middleware. The service iterates the three configured Google client identifiers and accepts the first that verifies; if none do, the login fails. The user record is then upserted by email address — updating `last_login_at` if the row exists, inserting it if not — and a token is signed.

The seventy-day expiry is a deliberate trade against security. A devotional application that demands re-authentication weekly will be abandoned; one that never expires a token cannot revoke it. Seventy days is the compromise, and it is a weakness recorded in Section 3.8.

2.3.2 Conversation module

This module is the system. It exposes three endpoints — a voice turn, a text turn and a paginated history — and the two turn endpoints share a single `generateReply` helper that differs only in whether transcription runs.

The shared helper loads the feature flags, conditionally loads the previous turns, selects the system prompt for the requested language, calls the model under a timer, conditionally synthesises speech under a second timer, dispatches a fire-and-forget write of all four timings, and returns.

The word *conditionally* is doing real work in that sentence. If speech synthesis is disabled by flag, or if the ElevenLabs call fails, the synthesis function returns `null` rather than raising, and the user receives a text-only reply instead of an error. If the context feature is disabled, the model is called with no history. The pipeline degrades in stages rather than failing outright.

2.3.3 Prompt engineering

The persona lives in a roughly ninety-line system prompt parameterised on language. It specifies the reply language, the form of address, the vocabulary of emotional tags available to the synthesiser with worked examples in both languages, a four-part response structure, a two-to-four sentence length target, and a set of safety constraints: defer to human pastoral care for matters of crisis, introduce no new revelation, and stay within orthodox doctrine.

The length target is not honoured in practice. Figure 8 shows a reply running to three paragraphs. Section 3.7 records this as a defect, and Section 3.5 connects it directly to the latency result — synthesis time scales with output length, and the model is producing four to five times the specified length.

2.3.4 Subscription and payments module

A subscription is created against Razorpay with a twelve-cycle count and persisted locally. Reading the current subscription refreshes it from Razorpay first and falls back to the stored row if that call fails, so a Razorpay outage degrades to stale data rather than an error.

Webhooks are the reconciliation path. Ten `subscription.*` event types are handled. Every webhook — valid or not — is recorded to an audit table along with the boolean result of its signature check.

2.3.5 Bible module

The Bible reader is served by a third-party public API at `bible.helloao.org`, not by this project's backend. It provides sixty-six books across six translations, two of them Telugu. The client wraps it with a thirty-second timeout, three retries with backoff, a connectivity pre-check and a SQLite cache. Reading position is saved per book and translation and restored on return.

2.3.6 Hymn library module

Twelve public-domain hymns are bundled with the application as approximately forty-second excerpts at 64 kbps mono, each with a public-domain painting as artwork. The songs endpoint returns a server-managed catalogue; if it returns nothing or fails, the client falls back to the bundled set, so the library works with no network at all.

The licensing of these assets carries an obligation the application does not currently meet; Appendix E and Section 3.8 record it.

2.3.7 Administrative console

The console is one HTML file with inline styles and inline JavaScript. It has no build step, no framework and no content-delivery-network dependency — a deliberate choice so that a venue's network cannot blank the dashboard during a demonstration. Charts are hand-drawn inline SVG for the same reason.

It presents six stat cards, a conversations-per-day chart, a language split, the per-stage latency breakdown, and six tabs: users, songs, conversations, webhooks, feature flags and the audit trail. It also ships a demonstration mode that serves fixtures entirely client-side behind a visible `DEMO DATA` badge. That mode is the origin of the latency discrepancy discussed in Section 3.5.

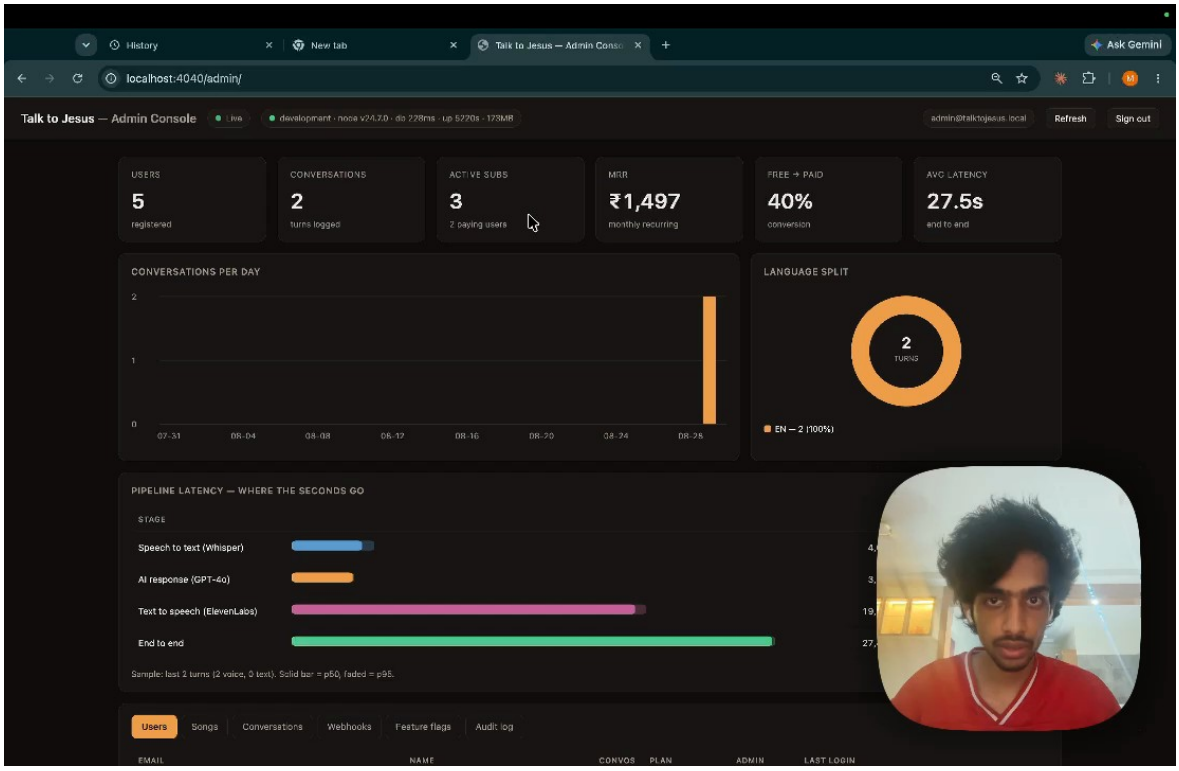

Figure 24 — The administrative console against the live instance. Five registered users, two logged turns, and a 27.5-second average end-to-end latency.

2.3.8 Feature flag module

Five flags are stored in a `feature_flags` table and read on every conversational turn behind a fifteen-second in-process cache. Reads never raise; on any error the module returns compiled-in defaults.

Table 6 — Runtime feature flags

Key	Default	Effect
`free_tier_limit`	3	Conversations allowed before the paywall
`maintenance_mode`	false	Refuses conversational turns with a friendly message
`tts_enabled`	true	When false, replies are text-only
`multi_turn_enabled`	true	When false, each turn is generated without history

Key	Default	Effect
`multi_turn_window`	6	How many previous turns are supplied as context

2.4 Key algorithms and logic

2.4.1 The timed pipeline

The instrumentation is the point, so the ordering matters.

```
FUNCTION generateReply(user, text, language):
    flags   ← getFeatureFlags()              -- 15 s cache, never
raises
    IF flags.maintenance_mode: ABORT 503

    history ← []
    IF flags.multi_turn_enabled:
        history ← getRecentTurns(user.id, flags.multi_turn_window)

    t0 ← now()
    reply ← callOpenAI(text, systemPrompt(language), history)
    llm_ms ← now() - t0

    audio, tts_ms ← null, 0
    IF flags.tts_enabled:
        t1 ← now()
        audio ← generateSpeech(reply)        -- returns null on
failure
        tts_ms ← now() - t1

    logConversationTurn(...)                 -- fire and forget, not
awaited
    RETURN reply, audio, llm_ms, tts_ms
```

Two properties are deliberate. The log write is dispatched without being awaited and swallows its own errors, so instrumentation can never slow or break a reply. And

`generateSpeech` returns `null` rather than raising, so a synthesis failure costs the user their audio but not their answer.

2.4.2 Language handling

The language parameter selects the system prompt and the emotional-tag vocabulary. It is **not** forwarded to Whisper.

The reason is code-switching. A Telugu speaker asking a question about a software project will say “capstone project” and “computer science” in English inside a Telugu sentence — Figure 8 shows exactly this pattern. Pinning Whisper to Telugu degrades those English tokens; auto-detection handles the mixture better. The cost is that a user who has selected Telugu but speaks English will be transcribed as English and receive a Telugu reply, because the prompt language and the detected language are independent. This is accepted behaviour, not a defect.

2.4.3 Emotional tag mapping

ElevenLabs accepts inline bracketed direction such as `[gently]` or `[prayerfully]`. The synthesis service checks whether the model’s output already carries such tags. If it does, they pass through. If not, the service inspects the text for Telugu keywords and prepends matching tags — for instance ప్రేమ or బిడ్డ maps to a loving and caring register, ప్రార్థన or ఆశీర్వాద to a prayerful and reverent one — defaulting to `[warmly] [gentle]`.

These tags are markup for the speech engine. They are not intended for display. Section 3.7 records that they are nevertheless displayed.

2.4.4 Multi-turn context

This is the mechanism that most invites overstatement, so it is stated plainly.

```
FUNCTION getRecentTurns(userId, limit):
    rows ← SELECT user_message, assistant_text FROM
conversation_logs
           WHERE user_id = userId
           ORDER BY created_at DESC LIMIT limit
    RETURN reverse(rows)                    -- oldest first, as the
model expects
```

Those turns are flattened into alternating user and assistant messages and prepended to the current message. **There is no retrieval, no embedding, no vector index and no semantic search.** The system does not retrieve relevant scripture; the model produces citations from its own parameters. Describing this as retrieval-augmented generation would be inaccurate, and the distinction matters for any assessment of whether citations can be trusted — Section 3.8 records that citation accuracy was not verified.

The function returns an empty list on any error, so a database problem costs context but not the conversation.

2.4.5 Entitlement resolution

The paywall is more involved than a boolean because payment providers are eventually consistent.

```
FUNCTION hasActiveSubscription(userId, freeLimit):
    user ← SELECT conversation_count FROM users WHERE id = userId
    IF user.conversation_count < freeLimit: RETURN true    -- free
tier

    subs ← SELECT * FROM subscriptions WHERE user_id = userId
    best ← pick by status priority: active > authenticated > created

    CASE best.status:
        'active':        RETURN true
        'authenticated': RETURN true         -- mandate approved,
first charge pending
        'created':       RETURN (now - best.created_at) < 24 hours
        otherwise:       RETURN false
```

The twenty-four-hour grace period on `created` exists because a user who has completed checkout may reach the application before Razorpay's webhook does. Without it, a paying user would be refused service in the seconds after paying. With it, an abandoned checkout grants a day of unpaid access. The trade favours the paying user, and Section 3.8 records the cost.

Note also that `incrementConversationCount` is a read-modify-write without a database-level atomic increment, so two truly concurrent turns from one account can lose an increment. Section 3.7 records this.

2.4.6 Webhook signature verification

```
FUNCTION verifySignature(rawBody, receivedSignature, secret):
    expected ← HMAC_SHA256(rawBody, secret)
    IF length(expected) ≠ length(receivedSignature): RETURN false
    RETURN timingSafeEqual(expected, receivedSignature)
```

The length check precedes the comparison because `timingSafeEqual` raises on unequal lengths rather than returning false. The comparison itself is constant-time, so response timing does not leak how much of a forged signature was correct.

Every webhook is recorded whether or not it verifies. Recording the failures is what makes it possible to demonstrate that verification is running — an audit table containing only successes is consistent with a system that never checks.

2.5 Screenshots

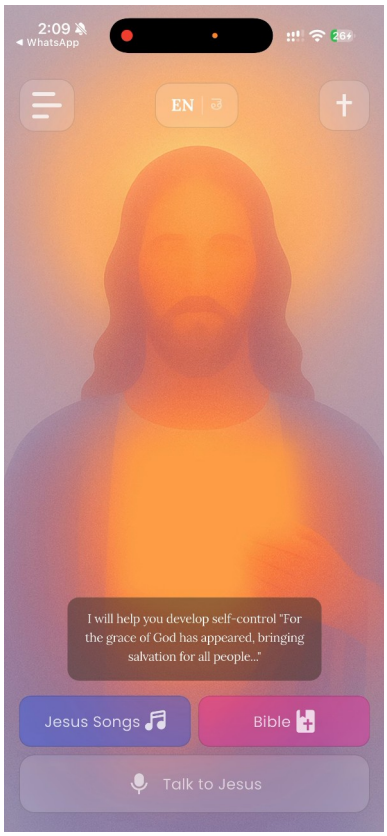

Figure 3 — Home screen in English. The verse card, the two navigation buttons and the voice bar.

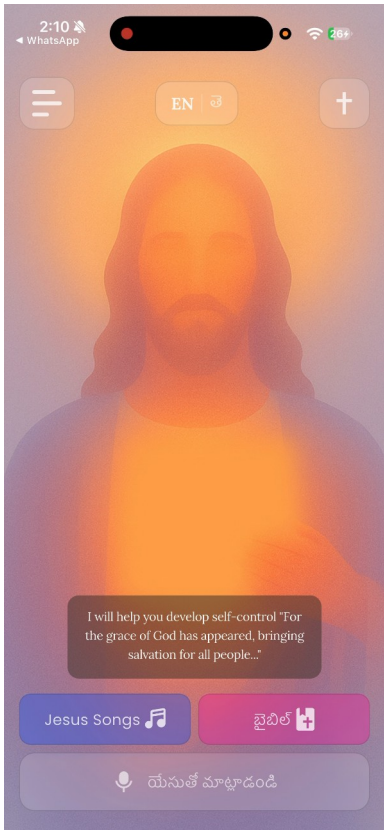

Figure 5 — The same screen in Telugu. Every visible string is translated, including the voice bar and the Bible button, and the switch takes effect without restarting the application.

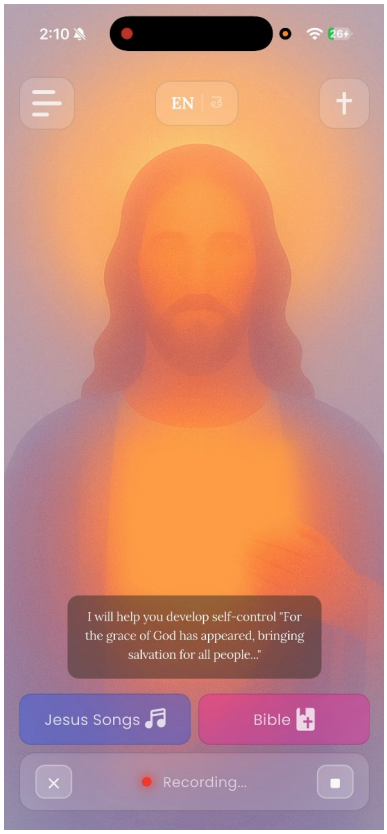

Figure 6 — Recording. The bar becomes a cancel control, a pulsing indicator and a stop control.

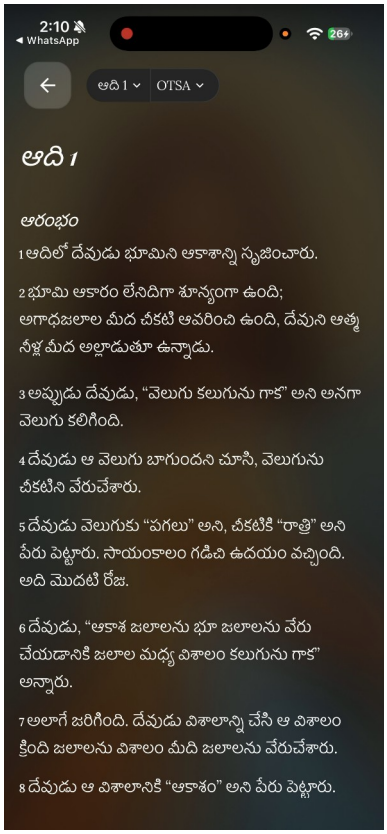

Figure 9 — The Bible reader in Telugu, showing Genesis 1 in the OTSA translation.

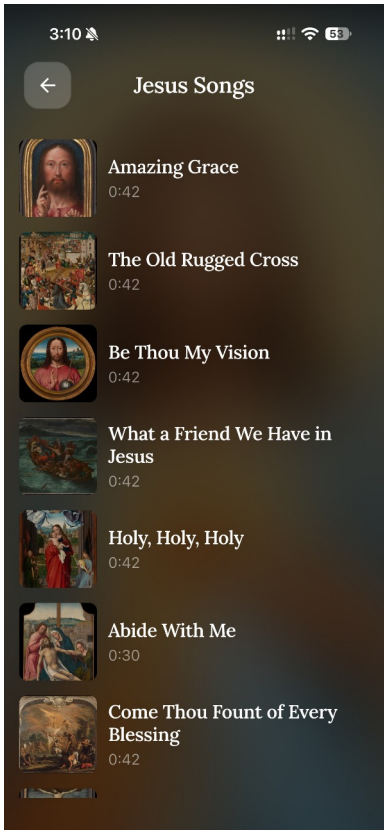

Figure 13 — The hymn library. Twelve public-domain hymns with public-domain artwork, bundled with the application and playable with no network.

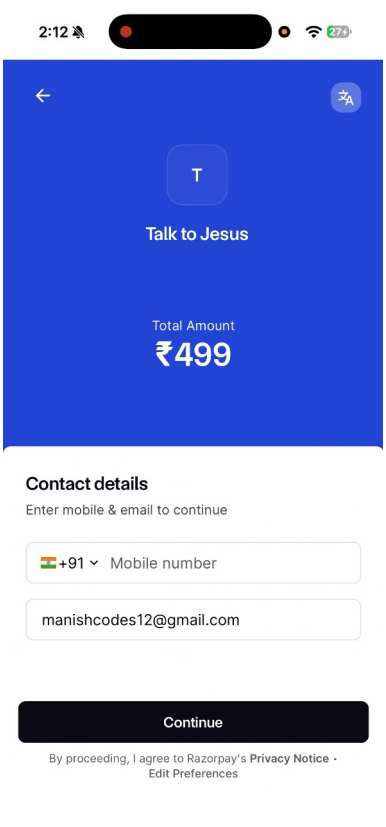

Figure 16 — Razorpay checkout at the ₹499 monthly plan.

CHAPTER 3: TESTING, VALIDATION & RESULTS

3.1 Test plan

3.1.1 Strategy

Testing operates at three levels, and the report is explicit about what each level can and cannot establish.

Automated unit tests cover pure logic and the decision points where a mistake is silent — entitlement resolution, signature verification, percentile arithmetic, token expiry, and the parsing of every model that crosses the network boundary. These run in continuous integration on every push and are the only level that runs unattended.

Manual functional verification covers the paths that cross a real network to a paid third party. The complete subscription flow — plan selection, Razorpay checkout, UPI authorisation, webhook reconciliation and the resulting change of application state — cannot be exercised automatically without either mocking away the thing under test or spending real

money on every continuous integration run. It was therefore executed by hand on physical hardware and recorded; Figures 15 through 18 are frames from that recording.

Instrumented measurement covers performance. Rather than timing the pipeline externally, the application records the duration of each stage of every turn into the database, and the administrative console reports the percentiles. This is the only level that produced a result contradicting the project's own prior documentation.

3.1.2 Tools

Layer	Tool	Invocation
Backend unit	Jest 30 with ts-jest	`npm test -- --ci`
Frontend unit	`flutter_test`	`flutter test`
Static analysis	`tsc --noEmit` via build; `flutter analyze`	in CI
Continuous integration	GitHub Actions	on push and pull request to `main`
Container verification	Docker	asserts built artefacts exist in the image
Measurement	In-application instrumentation, read via the console	continuous

The backend suite is hermetic. A setup file stubs every environment variable the modules read at import time, so the suite requires no credentials and no network. This was not the original state: an earlier version passed locally and failed in continuous integration precisely because module-level initialisation read variables that existed only on the developer's machine.

3.2 Requirement verification

3.2.1 Functional requirements

All twenty functional requirements defined in Phase 2 are implemented. Two carry qualifications.

Table 1 — Functional requirements and their implementation status

ID	Requirement (abbreviated)	Status	Evidence
FR1	Google OAuth sign-in across web, iOS and Android client IDs	Met	Figures 1–2; BE-052 to BE-054
FR2	JWT with configurable expiry	Met	BE-041 to BE-047
FR3	Create user on first login, update last-login after	Met	BE-053
FR4	Endpoint returning the authenticated user's profile	Met	`GET /api/user/me`
FR5	Voice upload, multipart, ≤ 10 MB, with language parameter	Met	Figure 6
FR6	Whisper transcription with automatic language detection	Met	Section 2.4.2
FR7	GPT-4o reply under a language-specific persona prompt	Met	Figure 8
FR8	ElevenLabs synthesis with emotional tags	Met, with defect	Figure 8; D-01 in Section 3.7
FR9	Single response carrying transcript, text and base64	Met	Section 2.1.2

ID	Requirement (abbreviated)	Status	Evidence
	audio		
FR10	Per-user conversation count with a three-conversation free tier	Met	BE-021 to BE-033
FR11	Plans endpoint filtered by environment	Met	Figure 15
FR12	Razorpay subscription creation	Met	Figures 16–17
FR13	Entitlement check by status with a grace period	Met	BE-021 to BE-033
FR14	Webhook handling with HMAC-SHA256 verification	Met	Figure 21; BE-034 to BE-040
FR15	Cancellation at end of billing cycle	Met	`POST /api/subscription/cancel`
FR16	Songs endpoint with pagination and search	Met	BE-056 to BE-060
FR17	Audio player with play, pause, seek and artwork	Partially met	Figure 14; previous and next are unimplemented stubs
FR18	Bible reading with navigation, translations and caching	Met	Figures 9–12

ID	Requirement (abbreviated)	Status	Evidence
FR19	Application-wide language switching without restart	Met	Figures 3 and 5; FE-001 to FE-006
FR20	Language parameter drives prompt and emotional expression	Met	Section 2.4.3

FR17 is recorded as partially met: the player renders previous and next controls, but both are unimplemented and emit only an analytics event. A control that appears functional and does nothing is a defect rather than an omission, and it is listed as such in Section 3.7.

3.2.2 Non-functional requirements

This table is where the project's honest assessment lives.

Table 2 — Non-functional requirements and their verification status

ID	Requirement (abbreviated)	Status	Basis
NFR1	Full pipeline within 5 seconds under normal load	**Not met**	Measured 27,457 ms p50. Section 3.5
NFR2	Music playback begins within 2 seconds	Met	Observed on device
NFR3	Launch to interactive under 3 seconds	Met	Observed on device
NFR4	Protected endpoints require a valid JWT	Met	BE-006 to BE-011
NFR5	Webhook HMAC-SHA256 with timing-safe	Met	BE-048 to BE-051

ID	Requirement (abbreviated)	Status	Basis
	comparison		
NFR6	Secrets in environment variables, never hardcoded	**Not met**	A long-lived JWT, a PostHog key and a Sentry DSN are committed. Section 3.8
NFR7	Row-level security enforced on all tables	Partially met	RLS is enabled, but no policies are defined and the API's service-tier key bypasses it. Section 2.1.1
NFR8	Material Design 3 with animated feedback	Met	Figures 3–18
NFR9	High contrast, focus management, text scaling 0.8×–1.4×	Partially met	Scaling and contrast are implemented; screen-reader optimisation is not complete
NFR10	Language switch updates all UI without restart	Met	Figures 3 and 5
NFR11	Cloud Run autoscaling from zero	Met	Section 4.2
NFR12	Stateless API permitting horizontal scaling	Met	Section 2.1.1
NFR13	Schema supports	Met	`language` is a free

ID	Requirement (abbreviated)	Status	Basis
	additional languages without change		text column
NFR14	Clean Architecture on the client	Met	`core` / `data` / `domain` / `presentation`
NFR15	Layered controllers, services, routes, middleware on the server	Met	Section 2.3
NFR16	TypeScript for compile-time safety	Met	`strict` mode; build gates CI
NFR17	Structured logging with per-environment levels	Met	Winston with `LOG_LEVEL`
NFR18	Sentry captures runtime errors	Partially met	Initialised, but the flavour configuration is hardcoded rather than read from the build-time defines
NFR19	PostHog tracks key product events	Met	Initialised and firing

Three requirements are not met and three are partially met. NFR1 is the significant one and is treated separately in Section 3.5.

3.3 Automated test results

Both suites were executed for this report. The captured output is in `docs/evidence/`.

Table 7 — Automated test suite composition

Tier	File	Cases	Area
Backend	admin.middleware.test.ts	5	Administrative access gate
Backend	consoleAdmin.auth.test.ts	4	Console administrator identity
Backend	jwt.test.ts	7	Token signing, expiry and verification
Backend	webhook.service.test.ts	6	Razorpay event handling
Backend	adminStats.service.test.ts	14	Percentile, revenue and conversion arithmetic
Backend	auth.middleware.test.ts	6	Bearer token authentication
Backend	song.service.test.ts	5	Catalogue pagination and search
Backend	razorpay.test.ts	4	HMAC signature verification
Backend	subscription.service.test.ts	12	Entitlement resolution and grace period
Backend	auth.service.test.ts	3	Google token verification and user upsert
Frontend	app_strings_test.dart	6	Bilingual string-table completeness

Tier	File	Cases	Area
Frontend	`conversation_response_test.dart`	4	Conversation response parsing
Frontend	`app_state_provider_test.dart`	10	Application state and language switching
Frontend	`bible_cache_test.dart`	8	Cache entry expiry and LRU accounting
Frontend	`subscription_model_test.dart`	13	Subscription status matrix
Frontend	`user_model_test.dart`	13	User parsing including the admin flag
Frontend	`bible_data_test.dart`	7	Book and translation reference data
Frontend	`song_test.dart`	7	Song value semantics
Frontend	`plan_model_test.dart`	7	Paise-to-rupee conversion and formatting
Total	**19 files**	**141**	

Backend: `Test Suites: 10 passed, 10 total. Tests: 66 passed, 66 total. Time: 2.045 s.`

Frontend: `+75: All tests passed!` across 9 files.

Total: 141 automated tests, 141 passing, none skipped, none failing.

A note on this figure, because it differs from every other document in the project. `TESTING.md`, the demonstration script and the presentation deck all state 62 backend tests and 137 in total. That was correct when written and is now stale: four cases covering the console administrator identity were added subsequently. The Phase 3 document states 40 backend tests with two failing, which is further out of date still. The number to quote is **141**.

Table 8 — Full automated test case list

ID	Suite	Case	Result
BE-001	`admin.middleware.test.ts`	Admin Middleware - should return 401 when no user is attached to the request	Pass
BE-002	`admin.middleware.test.ts`	Admin Middleware - should return 404 when the user is not an admin	Pass
BE-003	`admin.middleware.test.ts`	Admin Middleware - should return 404 when is_admin is missing entirely	Pass
BE-004	`admin.middleware.test.ts`	Admin Middleware - should reject a truthy-but-not-true is_admin value	Pass
BE-005	`admin.middleware.test.ts`	Admin Middleware - should call next() for an admin user	Pass
BE-006	`consoleAdmin.auth.test.ts`	Console admin authentication - authenticates a console token without reading the users table	Pass
BE-007	`consoleAdmin.auth.test.ts`	Console admin authentication - passes the admin gate once	Pass

ID	Suite	Case	Result
		authenticated	
BE-008	`consoleAdmin.auth.test.ts`	Console admin authentication - rejects a console token once ADMIN_EMAIL is removed	Pass
BE-009	`consoleAdmin.auth.test.ts`	Console admin authentication - does not let a forged claim through without a valid signature	Pass
BE-010	`jwt.test.ts`	JWT Utility - signToken - should return a valid JWT string	Pass
BE-011	`jwt.test.ts`	JWT Utility - signToken - should embed the payload in the token	Pass
BE-012	`jwt.test.ts`	JWT Utility - signToken - should set expiration to 70 days	Pass
BE-013	`jwt.test.ts`	JWT Utility - verifyToken - should return decoded payload for a valid token	Pass
BE-014	`jwt.test.ts`	JWT Utility -	Pass

ID	Suite	Case	Result
		verifyToken - should return null for an invalid token	
BE-015	`jwt.test.ts`	JWT Utility - verifyToken - should return null for a token signed with a different secret	Pass
BE-016	`jwt.test.ts`	JWT Utility - verifyToken - should return null for an expired token	Pass
BE-017	`webhook.service.test.ts`	Webhook Service - should handle subscription.charged and set last_charged_at	Pass
BE-018	`webhook.service.test.ts`	Webhook Service - should handle subscription.cancelled	Pass
BE-019	`webhook.service.test.ts`	Webhook Service - should handle subscription.authenticated and set last_charged_at from current_start	Pass
BE-020	`webhook.service.test.ts`	Webhook Service - should ignore non-subscription events gracefully	Pass

ID	Suite	Case	Result
BE-021	webhook.service.test.ts	Webhook Service - should handle missing subscription data in payload	Pass
BE-022	webhook.service.test.ts	Webhook Service - should handle subscription.activated and set last_charged_at as fallback	Pass
BE-023	adminStats.service.test.ts	Admin Stats Service - percentile - returns null for an empty set	Pass
BE-024	adminStats.service.test.ts	Admin Stats Service - percentile - returns the only value for a single sample	Pass
BE-025	adminStats.service.test.ts	Admin Stats Service - percentile - computes the median of an odd-length set	Pass
BE-026	adminStats.service.test.ts	Admin Stats Service - percentile - interpolates the median of an even-length set	Pass
BE-027	adminStats.service.test.ts	Admin Stats Service - percentile - computes p95 near the top of the range	Pass
BE-028	adminStats.serv	Admin Stats Service	Pass

ID	Suite	Case	Result
	ice.test.ts	- percentile - does not mutate the input array	
BE-029	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - returns 0 with no subscriptions	Pass
BE-030	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - converts paise to rupees	Pass
BE-031	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - sums across subscriptions	Pass
BE-032	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - treats a missing or null plan as zero rather than throwing	Pass
BE-033	adminStats.service.test.ts	Admin Stats Service - computeConversionRate - returns 0 when there are no users, without dividing by zero	Pass

ID	Suite	Case	Result
BE-034	`adminStats.service.test.ts`	Admin Stats Service - computeConversionRate - computes a percentage to one decimal place	Pass
BE-035	`adminStats.service.test.ts`	Admin Stats Service - computeConversionRate - returns 100 when every user pays	Pass
BE-036	`adminStats.service.test.ts`	Admin Stats Service - computeConversionRate - returns 0 when nobody pays	Pass
BE-037	`auth.middleware.test.ts`	Auth Middleware - should return 401 when no authorization header	Pass
BE-038	`auth.middleware.test.ts`	Auth Middleware - should return 401 when authorization header is not Bearer	Pass
BE-039	`auth.middleware.test.ts`	Auth Middleware - should return 401 when JWT token is invalid	Pass
BE-040	`auth.middleware.test.ts`	Auth Middleware - should return 401	Pass

ID	Suite	Case	Result
		when user is not found in database	
BE-041	auth.middleware.test.ts	Auth Middleware - should set req.user and call next() on valid auth	Pass
BE-042	auth.middleware.test.ts	Auth Middleware - should return 500 on unexpected error	Pass
BE-043	song.service.test.ts	Song Service - should return paginated songs with correct offset	Pass
BE-044	song.service.test.ts	Song Service - should apply search filter when provided	Pass
BE-045	song.service.test.ts	Song Service - should not apply search filter when not provided	Pass
BE-046	song.service.test.ts	Song Service - should throw on database error	Pass
BE-047	song.service.test.ts	Song Service - should calculate correct offset for first page	Pass
BE-048	razorpay.test.ts	Razorpay Utility - verifyWebhookSignature - should return true for a valid	Pass

ID	Suite	Case	Result
		signature	
BE-049	`razorpay.test.ts`	Razorpay Utility - verifyWebhookSignature - should return false for an invalid signature	Pass
BE-050	`razorpay.test.ts`	Razorpay Utility - verifyWebhookSignature - should return false for a signature with wrong length	Pass
BE-051	`razorpay.test.ts`	Razorpay Utility - verifyWebhookSignature - should return false for tampered body	Pass
BE-052	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true when user is within free tier (count < 3)	Pass
BE-053	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should respect a custom free tier limit	Pass
BE-054	`subscription.service.test.ts`	Subscription Service - hasActiveSubscripti	Pass

ID	Suite	Case	Result
		on - should return false when user exceeds free tier and has no subscription	
BE-055	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for an active subscription	Pass
BE-056	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should prefer an active subscription over a stale created one	Pass
BE-057	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for an authenticated subscription	Pass
BE-058	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for a created subscription inside the 24h grace period	Pass

ID	Suite	Case	Result
BE-059	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return false for a created subscription past the 24h grace period	Pass
BE-060	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return false when user not found	Pass
BE-061	`subscription.service.test.ts`	Subscription Service - getUserSubscription - should return the latest subscription	Pass
BE-062	`subscription.service.test.ts`	Subscription Service - getUserSubscription - should return null when no subscription exists	Pass
BE-063	`subscription.service.test.ts`	Subscription Service - incrementConversationCount - should increment and return the new count	Pass
BE-064	`auth.service.te`	Auth Service -	Pass

ID	Suite	Case	Result
	st.ts	should create a new user when not found in DB	
BE-065	auth.service.test.ts	Auth Service - should return existing user and update last_login_at	Pass
BE-066	auth.service.test.ts	Auth Service - should throw on invalid Google token	Pass
FE-001	app_strings_test.dart	AppStrings every English key has a Telugu translation	Pass
FE-002	conversation_response_test.dart	ConversationResponse fromJson parses all fields	Pass
FE-003	app_strings_test.dart	AppStrings returns the translated string for each language	Pass
FE-004	conversation_response_test.dart	ConversationResponse fromJson handles null/missing fields with defaults	Pass
FE-005	app_strings_test.dart	AppStrings English and Telugu values actually differ	Pass
FE-006	conversation_response_test.dart	ConversationResponse toJson produces snake_case keys	Pass
FE-007	app_strings_test.dart	AppStrings falls back to the key itself	Pass

ID	Suite	Case	Result
		for an unknown key	
FE-008	conversation_response_test.dart	ConversationResponse fromJson/toJson round-trip	Pass
FE-009	app_strings_test.dart	AppStrings getAll returns a non-empty map for both languages	Pass
FE-010	app_strings_test.dart	AppStrings covers the screens used in the demo flow	Pass
FE-011	app_state_provider_test.dart	AppState has correct default values	Pass
FE-012	app_state_provider_test.dart	AppState copyWith updates only specified fields	Pass
FE-013	app_state_provider_test.dart	AppStateNotifier initial state has default values	Pass
FE-014	app_state_provider_test.dart	AppStateNotifier setLanguage updates the language	Pass
FE-015	app_state_provider_test.dart	AppStateNotifier toggleHighContrastMode toggles the mode	Pass
FE-016	app_state_provider_test.dart	AppStateNotifier setAudioPermission sets the permission flag	Pass
FE-017	app_state_provi	AppStateNotifier	Pass

ID	Suite	Case	Result
	der_test.dart	incrementCounter increases counter by 1	
FE-018	app_state_provider_test.dart	AppStateNotifier resetCounter sets counter back to 0	Pass
FE-019	app_state_provider_test.dart	AppLanguage english has correct code and displayName	Pass
FE-020	app_state_provider_test.dart	AppLanguage telugu has correct code and displayName	Pass
FE-021	bible_cache_test.dart	BibleCacheEntry toMap/fromMap round-trip preserves all fields	Pass
FE-022	subscription_model_test.dart	Subscription fromJson parses all fields correctly	Pass
FE-023	bible_cache_test.dart	BibleCacheEntry fromMap handles missing optional fields with defaults	Pass
FE-024	bible_cache_test.dart	BibleCacheEntry isExpired returns false for future expiry	Pass
FE-025	bible_cache_test.dart	BibleCacheEntry isExpired returns true for past expiry	Pass

ID	Suite	Case	Result
FE-026	`bible_cache_test.dart`	BibleCacheEntry copyWithAccess increments access count	Pass
FE-027	`bible_cache_test.dart`	BibleCacheEntry copyWith updates only specified fields	Pass
FE-028	`subscription_model_test.dart`	Subscription fromJson parses nested plan when present	Pass
FE-029	`bible_cache_test.dart`	ReadingPosition toMap/fromMap round-trip preserves all fields	Pass
FE-030	`bible_cache_test.dart`	ReadingPosition positionKey combines translationId and bookId	Pass
FE-031	`subscription_model_test.dart`	Subscription toJson produces correct keys	Pass
FE-032	`subscription_model_test.dart`	Subscription toJson includes plan when present	Pass
FE-033	`subscription_model_test.dart`	Subscription isActive returns true for active status	Pass
FE-034	`subscription_model_test.dart`	Subscription isActive returns true	Pass

ID	Suite	Case	Result
		for authenticated status	
FE-035	subscription_model_test.dart	Subscription isActive returns false for cancelled status	Pass
FE-036	subscription_model_test.dart	Subscription isCancelled returns true for cancelled status	Pass
FE-037	subscription_model_test.dart	Subscription isPaused returns true for paused status	Pass
FE-038	subscription_model_test.dart	Subscription isPastDue returns true for past_due status	Pass
FE-039	subscription_model_test.dart	CreateSubscriptionResponse shortUrl returns value from razorpaySubscription	Pass
FE-040	subscription_model_test.dart	CreateSubscriptionResponse shortUrl returns null when razorpaySubscription is null	Pass
FE-041	subscription_model_test.dart	CurrentSubscriptionResponse fromJson handles null subscription	Pass
FE-042	user_model_test	UserModel fromJson	Pass

ID	Suite	Case	Result
	.dart	parses all fields correctly	
FE-043	user_model_test .dart	UserModel fromJson handles null optional fields	Pass
FE-044	user_model_test .dart	UserModel toJson produces correct snake_case keys	Pass
FE-045	user_model_test .dart	UserModel fromJson/toJson round-trip preserves data	Pass
FE-046	user_model_test .dart	UserModel copyWith creates a new instance with updated fields	Pass
FE-047	user_model_test .dart	UserModel isAdmin defaults to false when the field is absent	Pass
FE-048	user_model_test .dart	UserModel isAdmin parses true from the backend	Pass
FE-049	user_model_test .dart	UserModel isAdmin survives a toJson/fromJson round-trip	Pass
FE-050	user_model_test .dart	UserModel copyWith can toggle isAdmin without touching other fields	Pass

ID	Suite	Case	Result
FE-051	`user_model_test.dart`	UserModel isTester returns true for tester user ID	Pass
FE-052	`user_model_test.dart`	UserModel isTester returns false for regular user	Pass
FE-053	`user_model_test.dart`	CreateOrGetUserResponse fromJson parses nested user and token	Pass
FE-054	`user_model_test.dart`	CreateOrGetUserResponse fromJson handles null token	Pass
FE-055	`bible_data_test.dart`	BibleData books list is not empty	Pass
FE-056	`bible_data_test.dart`	BibleData Genesis is the first book with 50 chapters	Pass
FE-057	`bible_data_test.dart`	BibleData Psalms has 150 chapters	Pass
FE-058	`bible_data_test.dart`	BibleData all books have positive chapter counts	Pass
FE-059	`bible_data_test.dart`	BibleData versions list contains expected translations	Pass
FE-060	`bible_data_test.dart`	BibleData versions list has 6 entries	Pass
FE-061	`bible_data_test.dart`	BibleBook can be constructed with name and	Pass

ID	Suite	Case	Result
		totalChapters	
FE-062	song_test.dart	Song copyWith updates specified fields only	Pass
FE-063	song_test.dart	Song copyWith with no args returns equal copy	Pass
FE-064	song_test.dart	Song equality works for identical songs	Pass
FE-065	song_test.dart	Song equality fails for different songs	Pass
FE-066	song_test.dart	Song hashCode is consistent for equal objects	Pass
FE-067	song_test.dart	Song toString includes all fields	Pass
FE-068	song_test.dart	Song handles null optional fields	Pass
FE-069	plan_model_test.dart	Plan fromJson parses all fields correctly	Pass
FE-070	plan_model_test.dart	Plan toJson produces correct keys	Pass
FE-071	plan_model_test.dart	Plan fromJson/toJson round-trip preserves data	Pass
FE-072	plan_model_test.dart	Plan priceInRupees converts paise to rupees	Pass
FE-073	plan_model_test	Plan priceInRupees	Pass

ID	Suite	Case	Result
	.dart	handles fractional amounts	
FE-074	plan_model_test .dart	Plan formattedPrice returns rupee symbol with amount	Pass
FE-075	plan_model_test .dart	Plan formattedPrice truncates decimals	Pass

3.4 Functional verification on device

The following was executed by hand on an iPhone against the live backend and recorded. Timestamps refer to the demonstration recording.

Table 9 — Functional verification on device

#	Scenario	Expected	Observed	Status
M-01	Sign in with Google	Account created, home screen shown	As expected	Pass
M-02	Open profile drawer	Name, address and navigation shown	As expected (00:05)	Pass
M-03	Switch to Telugu	All visible strings change without restart	As expected (00:27)	Pass
M-04	Read Genesis 1 in Telugu	Telugu text renders correctly	As expected (00:13)	Pass
M-05	Change translation	Six translations offered, selection applied	As expected (00:17)	Pass
M-06	Select book and	Searchable list,	As expected	Pass

#	Scenario	Expected	Observed	Status
	chapter	chapter grid	(00:25)	
M-07	Open hymn library	Twelve hymns with artwork	As expected	Pass
M-08	Play a hymn	Playback starts, seek bar advances	As expected	Pass
M-09	Use previous / next in the player	Track changes	**No effect**	**Fail — D-02**
M-10	Record a voice message	Recording indicator, cancel and stop	As expected (01:00)	Pass
M-11	Receive a spoken reply	Text and audio returned	As expected, after ~28 s	Pass, but see NFR1
M-12	Exceed the free tier	Paywall presented	As expected (02:37)	Pass
M-13	Complete Razorpay checkout by UPI	Payment succeeds	As expected (03:24)	Pass
M-14	Subscription state reflected in app	Badge turns active	As expected (03:36)	Pass
M-15	Open conversation history	Previous turns listed with language and age	As expected, with D-01	Pass, with defect
M-16	Administrative console as a non-admin	Access refused without confirming the	404 returned	Pass

#	Scenario	Expected	Observed	Status
		surface exists		

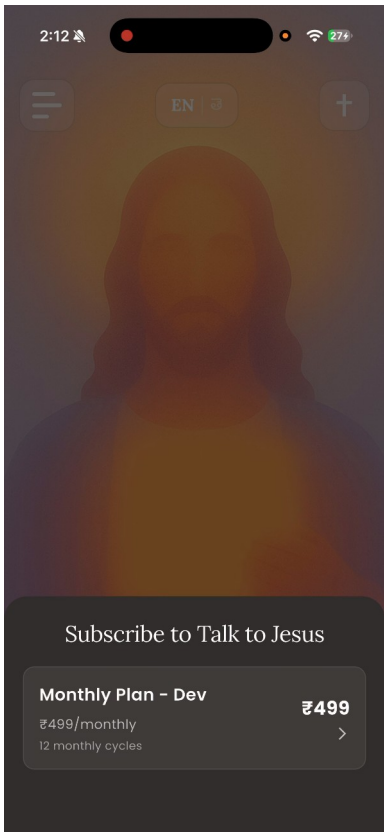

Figure 15 — The paywall after the free tier is exhausted.

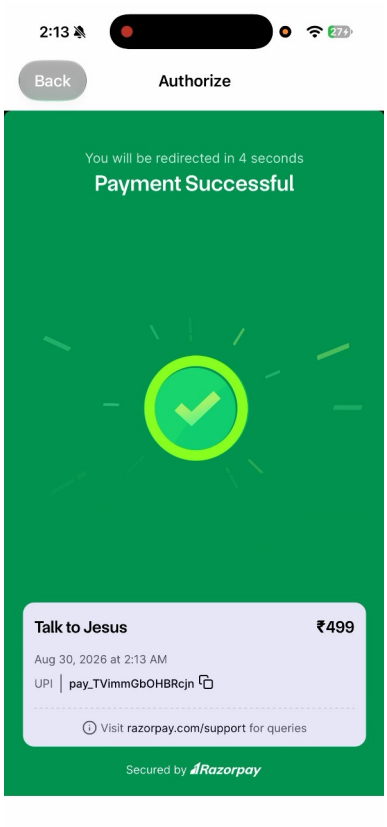

Figure 17 — Payment confirmed, with the Razorpay payment identifier.

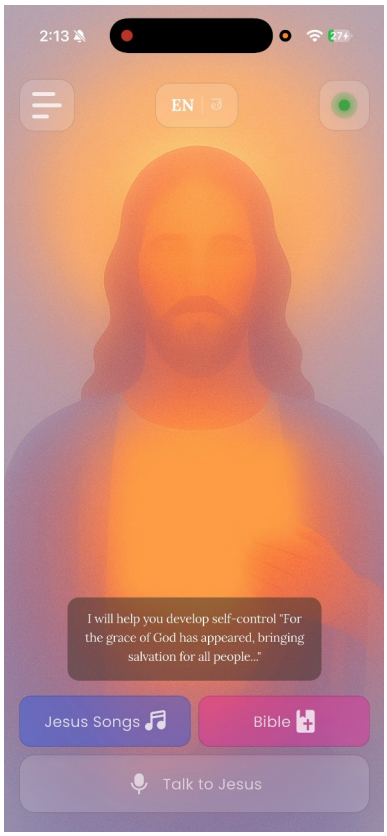

Figure 18 — The home screen after payment. The badge at the top right is now active.

3.5 The latency result

This is the project's principal measured finding, and it contradicts the project's own earlier documentation.

Table 10 — Measured pipeline latency, live instance

Stage	p50	p95	Share of p50 total
Speech to text (Whisper)	4,004 ms	4,715 ms	15 %
AI response (GPT-4o)	3,471 ms	3,546 ms	13 %
Text to speech (ElevenLabs)	19,618 ms	20,252 ms	**71 %**
End to end	27,457 ms	—	100 %

Sample: the last two turns, both voice, zero text.

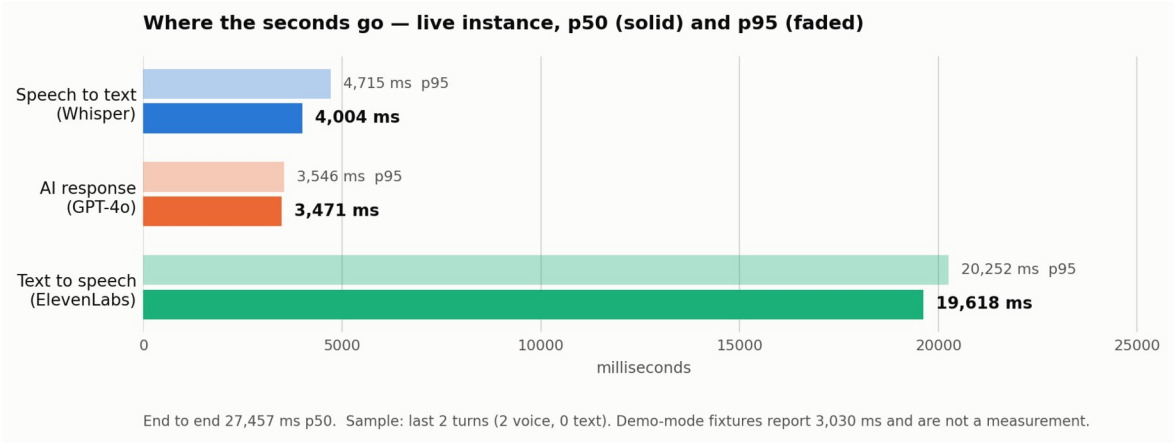

Figure 29 — Measured per-stage latency, live instance.

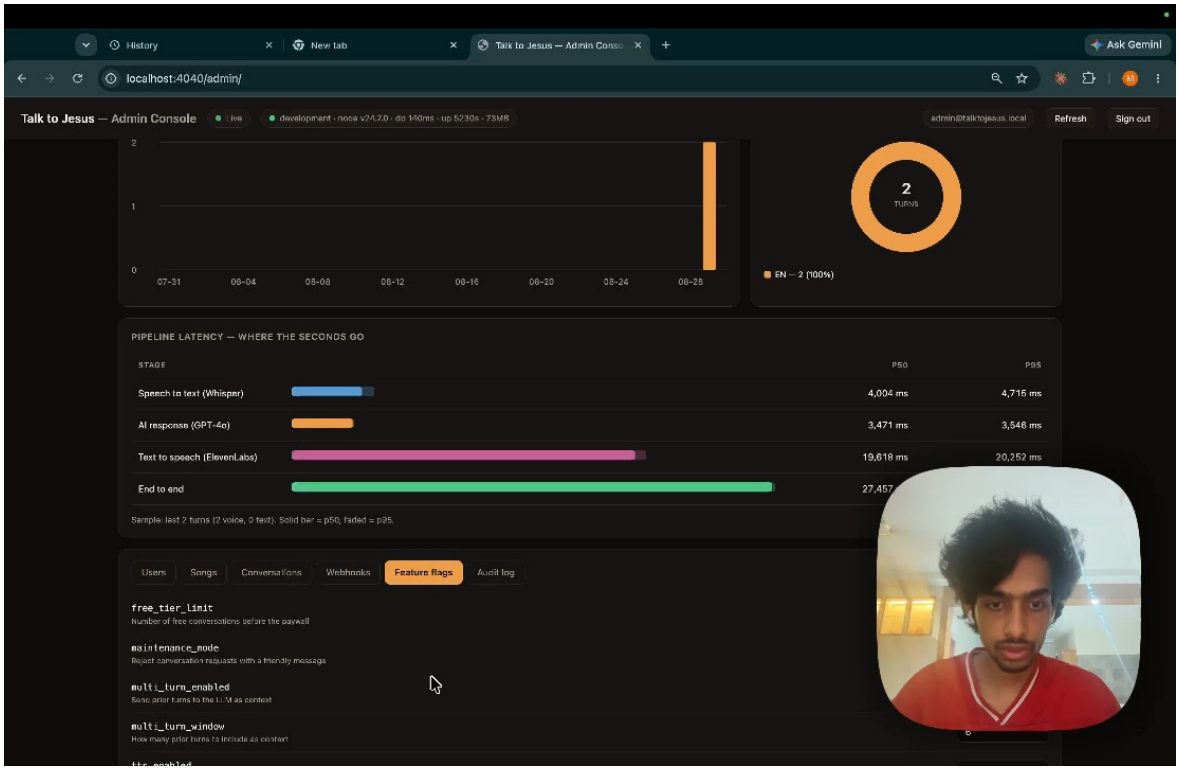

Figure 25 — The same measurement as reported by the administrative console.

Where the earlier figure came from. Phase 3 and the presentation deck state a three-to-six second pipeline. The administrative console's demonstration mode reports an end-to-end p50 of 3,030 ms over a fixture sample of 500 turns. That number is a hand-written fixture in the console's JavaScript. It was never a measurement, and the resemblance between it and the figure quoted in the earlier documents is the likely explanation for how the claim entered those documents.

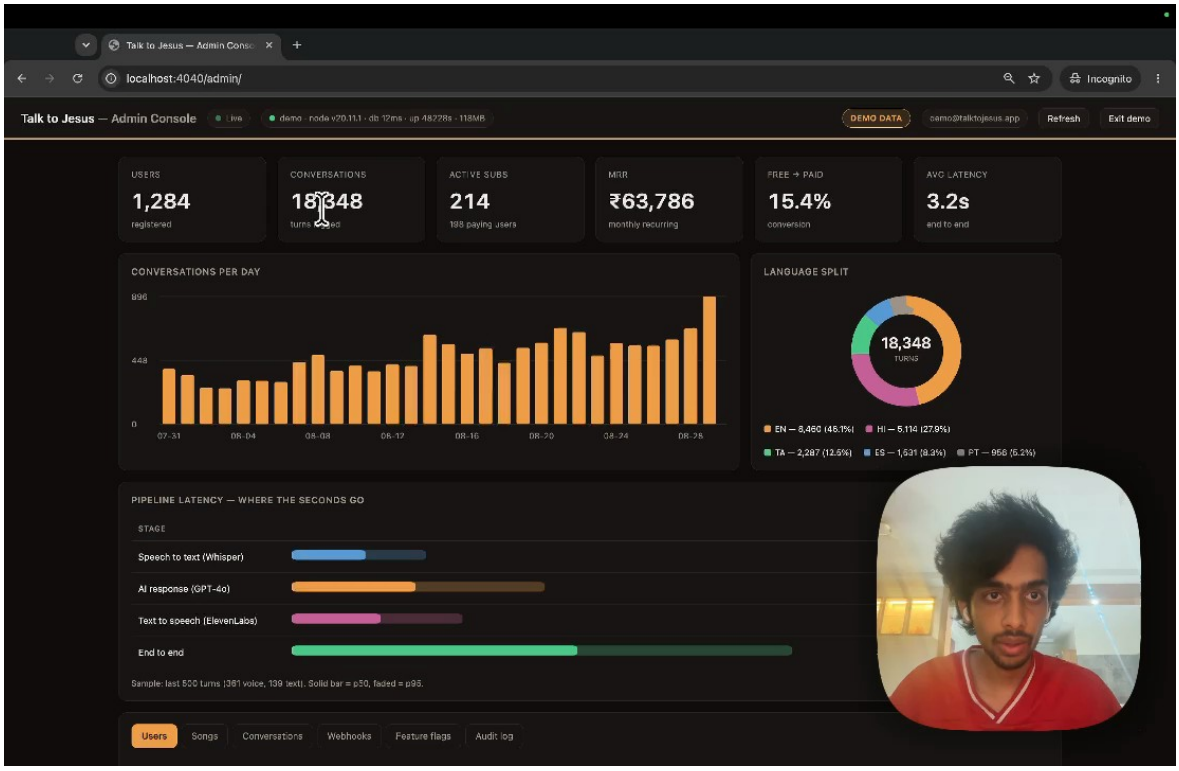

Figure 19 — Demonstration mode, showing the fixture data. The DEMO DATA badge is visible at the top right, and the end-to-end fixture reads 3,030 ms.

What the measurement means. Three observations follow, and they are separable.

First, the sample is two turns. Two turns cannot establish a distribution, and the p95 column is arithmetically meaningless at that sample size. What two turns *can* establish is an order of magnitude, and the order of magnitude is seconds-times-ten, not seconds. NFR1 specifies five seconds. The measurement is 27.5. No plausible sampling error closes a gap of that size.

Second, the distribution across stages is the useful part. Transcription and generation together account for 7.5 seconds; synthesis alone accounts for 19.6. Any optimisation effort directed at the model or the transcriber is misdirected. The latency is a speech synthesis problem.

Third, synthesis time scales with the length of the text being synthesised, and Section 2.3.3 records that the persona prompt specifies a two-to-four sentence reply while the model actually produces three paragraphs. The most probable single cause of the 19.6 seconds is therefore not the synthesis vendor but the prompt: the system is asking a speech engine to render four to five times more text than the design intended. Section 6.4 proposes the experiment that would confirm this.

The honest summary is that the system works and is too slow, that the instrumentation is what revealed it, and that the same instrumentation identifies where to look.

3.6 Correcting the Phase 3 document

Phase 3 was accurate when submitted. Several of its statements are no longer true, and one was incorrect at the time.

Table 13 — Corrections to the Phase 3 document

Phase 3 states	Current position
38 of 40 backend tests pass; 2 fail	66 of 66 pass across 10 suites
65 frontend tests across 8 files	75 across 9 files
Root cause of the failures: “the mock returns subscription data but the resolved user row evaluates to null or 0”	**Incorrect.** The test helper made only `.single()` awaitable, while the function under test awaits the query builder directly to obtain an array. Every “expect false” assertion was passing for the wrong reason
Conversation history and multi-turn context are future work	Both implemented
Cloud Run cold start approximately 2 s	Measured 4.4 s cold, ~0.4 s warm
Pipeline 3–6 s	Measured 27.5 s p50; the 3–6 s figure traces to demonstration fixtures
No mention of the administrative console, feature flags, webhook audit or CI	All four exist

The second row is worth dwelling on. The original diagnosis was plausible and wrong, and the tests it described as failing were, in the passing cases, succeeding for a reason unrelated to the behaviour they claimed to verify. A test that passes for the wrong reason is more dangerous than one that fails.

3.7 Defects identified

Table 12 — Defects identified

ID	Severity	Defect	Status
D-01	Medium	Emotional-control tags (`[gently]`, `[comfortingly]`, `[prayerfully]`) are rendered verbatim to the user in the conversation history and in the reply toast. They are markup for the speech engine and should be stripped before display	Open
D-02	Medium	The audio player's previous and next controls are unimplemented; they appear functional and emit only an analytics event	Open
D-03	Medium	Replies run to three paragraphs against a persona prompt specifying two to four sentences, which is the probable principal cause of the synthesis latency in Section 3.5	Open
D-04	Low	`incrementConversationCount` is a	Open

ID	Severity	Defect	Status
		read-modify-write with no atomic database increment; two concurrent turns from one account can lose an increment	
D-05	Low	The inspirational verse on the home screen is a hardcoded string rather than dynamic content	Open
D-06	Low	The connectivity provider is a stub that always reports connected; its own comment says so. The real connectivity check is used only inside the Bible repository	Open
D-07	Low	Two Bible cache implementations exist; only the smaller is wired in. The larger is dead code	Open
D-08	Low	The application has no about or credits screen, which the bundled hymn	Open

ID	Severity	Defect	Status
		licensing requires. See Appendix E	
D-09	Informational	The Android application identifier is still the scaffold default `com.example.talktojesus`, which would block a Play Store submission	Open

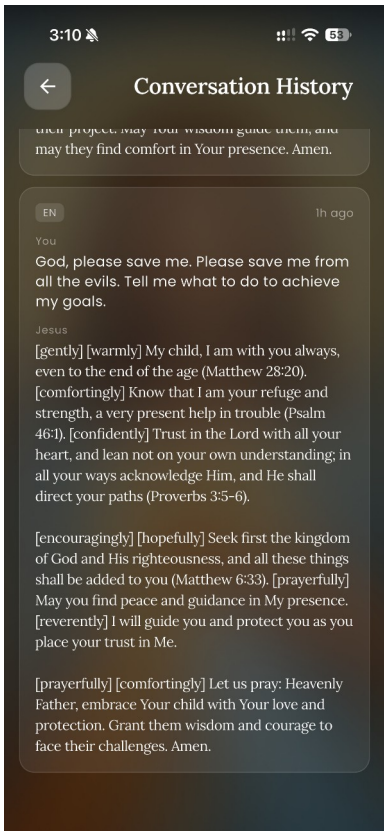

Figure 8 — Defect D-01. The bracketed tags are direction for the speech engine and were never intended to reach the reader.

3.8 Properties not verified

The following were not tested. They are listed because a report that omits them implies a coverage that does not exist.

Table 11 — Properties not verified

Not verified	Reason	Consequence
Scriptural accuracy of citations	Requires theological review beyond the scope of this project	The model generates citations from its parameters with no retrieval step (Section 2.4.4). A citation could be misattributed and nothing in the system would detect it
Behaviour under concurrent load	ElevenLabs quota made repeated load runs impractical	Throughput and the behaviour of the pipeline under contention are unknown
Telugu transcription accuracy in noise	No controlled acoustic test was performed	Degradation in a noisy environment is expected but unquantified
Controllers and the pipeline end to end	The automated suite covers services, middleware and utilities only	An error in request handling or response shaping would not be caught by the suite
Any user interface behaviour	No widget or integration tests exist	Every screen is verified only by manual inspection
Authorization isolation between users	No automated test asserts that one user cannot read another's conversations	Given that the database's row-level security is bypassed by the service key (Section 2.1.1), this is the single most valuable test the project does not have
Token revocation	Not implemented, so not tested	A seventy-day token cannot be invalidated before expiry; rotating the administrator password does not revoke

Not verified	Reason	Consequence
		tokens already issued
Rate limiting	Not implemented	No protection against abuse of the paid pipeline beyond the free-tier counter
Cross-origin restrictions	CORS is configured fully open	Any origin can call the API with a valid token
Secret hygiene in the repository	Reviewed, not enforced	A long-lived test JWT, a PostHog key and a Sentry DSN are committed to source control, and three `.env` backup files sit untracked in the working directory. These must be removed before any code submission
Personal data handling	No retention or redaction policy exists	Conversation transcripts are personal and sometimes sensitive; they are stored indefinitely with no deletion path

The authorization row deserves emphasis. Because the API holds a key that bypasses row-level security, every user-scoped query depends on application code filtering correctly. That property is currently guaranteed by inspection alone.

CHAPTER 4: EXECUTION / DEPLOYMENT DETAILS

4.1 Execution environment

Table 14 — Execution environment

Component	Configuration
Backend runtime, production	Node 20 on `node:20-alpine`, port 8080,

Component	Configuration
	`NODE_ENV=production`
Backend runtime, development	Node 24.7.0 on the development machine, port 4040
Hosting	Google Cloud Run, service `talktojesus-backend`, region `us-central1`, scales from zero
Image registry	Artifact Registry, `us-central1-docker.pkg.dev/talktojesus-backend/cloud-run-source-deploy`
Build	Cloud Build, machine type `E2_HIGHCPU_8`, 1200 s timeout
Database	Supabase-hosted PostgreSQL, eight tables
Mobile toolchain, CI	Flutter 3.38.6 stable
Mobile toolchain, development	Flutter 3.32.7
Continuous integration	GitHub Actions on push and pull request to `main`

The two version skews are worth noting rather than hiding. The backend is developed on Node 24 and deployed on Node 20; the mobile application is built locally on Flutter 3.32.7 and in continuous integration on 3.38.6. Neither has produced a defect, but both are latent risks, and reconciling them is listed in Section 6.4.

4.2 Deployment steps

4.2.1 Continuous integration

`.github/workflows/ci.yml` runs three jobs on every push and pull request to `main`, with in-progress runs cancelled when a newer commit arrives.

Backend. Install with `npm ci`, compile with `npm run build`, then `npm test -- --ci`. Compilation is a deliberate gate: because the project is `strict`, a type error fails the build before any test runs. No secrets are required, because the test setup file stubs every environment variable read at import time.

Frontend. `flutter pub get`, then `flutter analyze --no-fatal-infos --no-fatal-warnings`, then `flutter test`. Five pre-existing analyzer notices are tolerated by that invocation and are recorded here rather than silently suppressed.

Container. Build the image, then assert inside it that both `/app/dist/index.js` and `/app/public/admin/index.html` are present. The second assertion exists because its failure mode — a 404 at `/admin` — is invisible until someone opens the console.

4.2.2 Continuous deployment

Deployment is Cloud Build: build the image without cache, verify the same two artefacts, push to Artifact Registry tagged with the commit SHA, then update the Cloud Run service.

There is deliberately **no test gate in the deployment pipeline**. Tests run in GitHub Actions in parallel. The reasoning is that a deployment blocked by a flaky suite is worse than a deployment that ships alongside a failing test which the parallel pipeline will report within a minute. This is a defensible trade but it is a trade, and it means a red build does not by itself prevent a release.

One further caveat, recorded because it would otherwise mislead: the `cloudbuild.yaml` in the repository is not currently the file the live trigger uses. The trigger carries an equivalent configuration inline and therefore ignores the checked-in file. The two are equivalent today; nothing enforces that they remain so.

4.2.3 Local execution

Verified working. Full instructions are in Appendix B. In summary: provision the database with the three SQL files in order, populate the backend environment file, `npm ci && npm run dev` for the API on port 4040, and `flutter run --dart-define=API_BASE_URL=http://localhost:4040` for the client.

4.3 Demonstration screenshots

The complete set of twenty-nine figures, with sources and captions, is indexed at `docs/figures/FIGURES.md`. Figures 1 through 25 are drawn from the two demonstration recordings and from native device screenshots; figures 26 through 29 are generated from the codebase and from the measurements in Chapter 3.

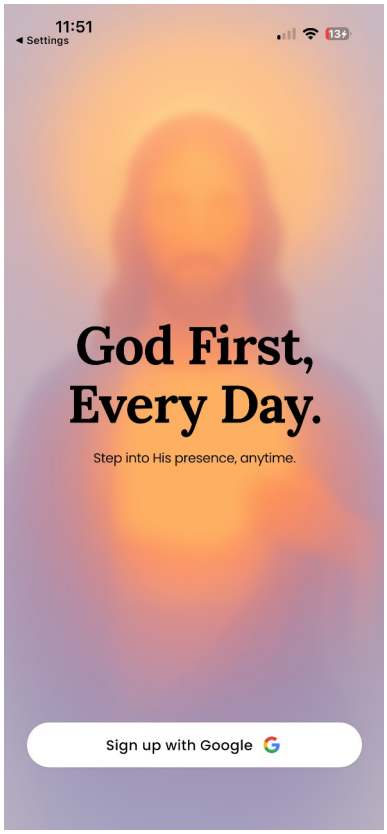

Figure 1 — The login screen.

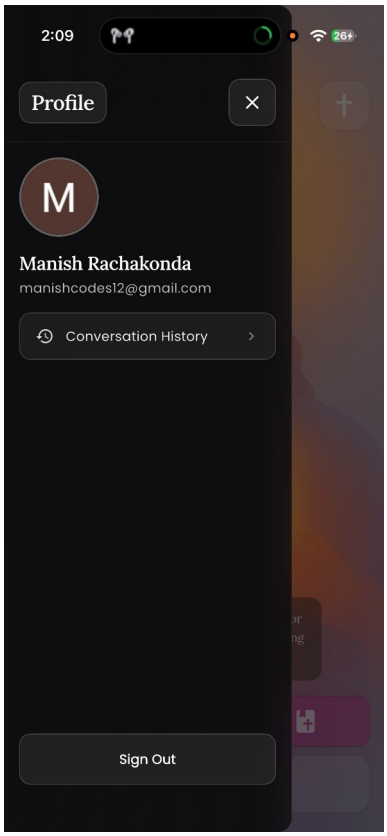

Figure 4 — The profile drawer, reached from the home screen.

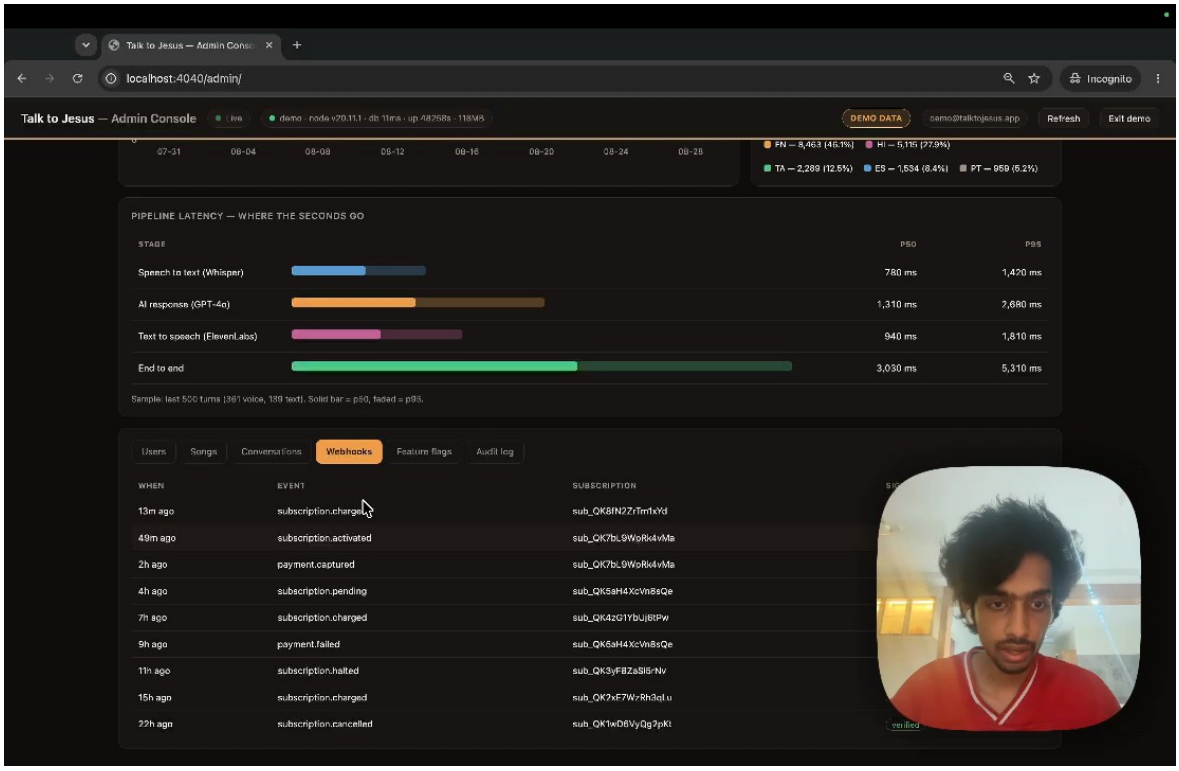

Figure 21 — The webhook audit tab. Signature verification status is recorded per event, including for events that fail.

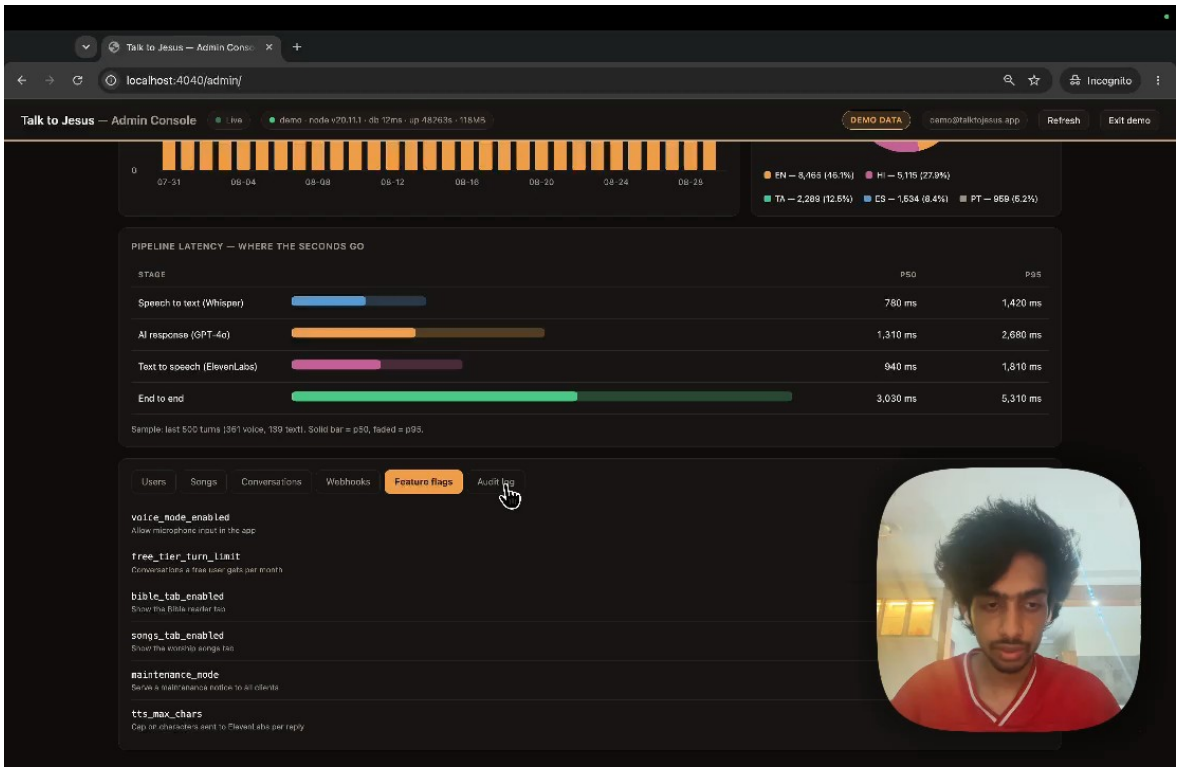

Figure 22 — Runtime feature flags, editable without redeployment.

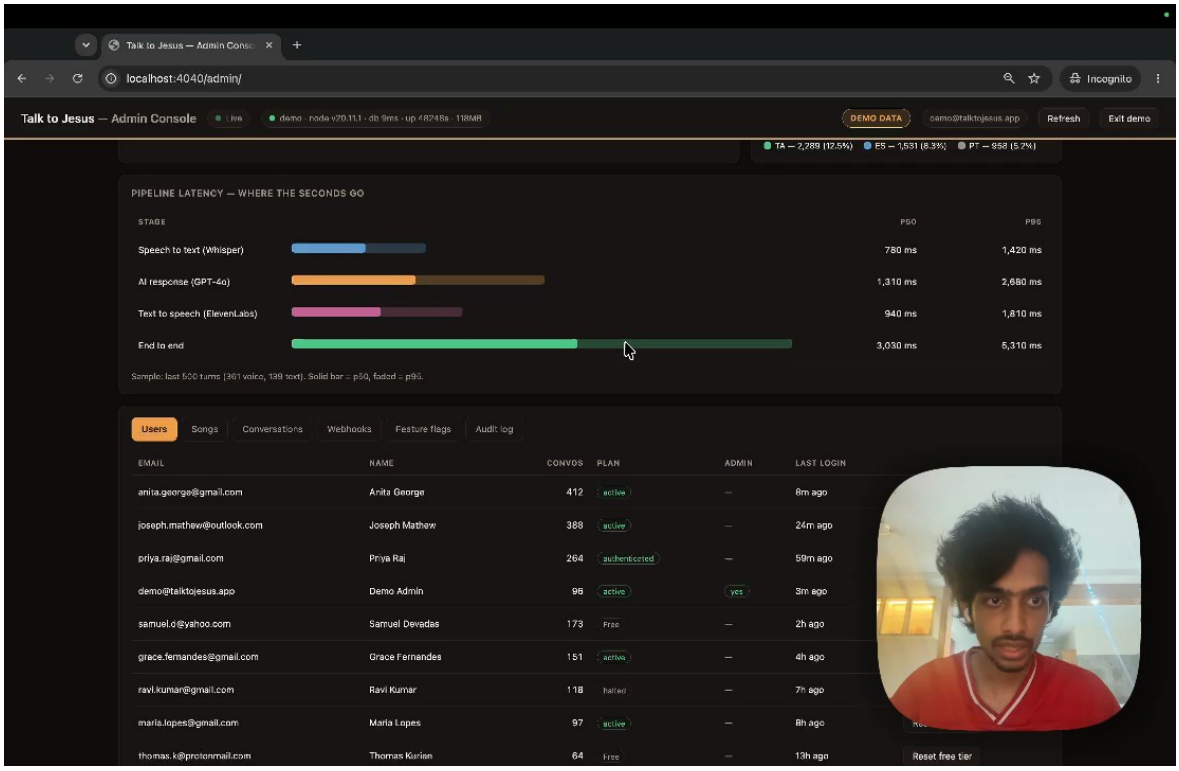

Figure 20 — The users tab. Shown in demonstration mode; the live tab displays real addresses and is therefore not reproduced here.

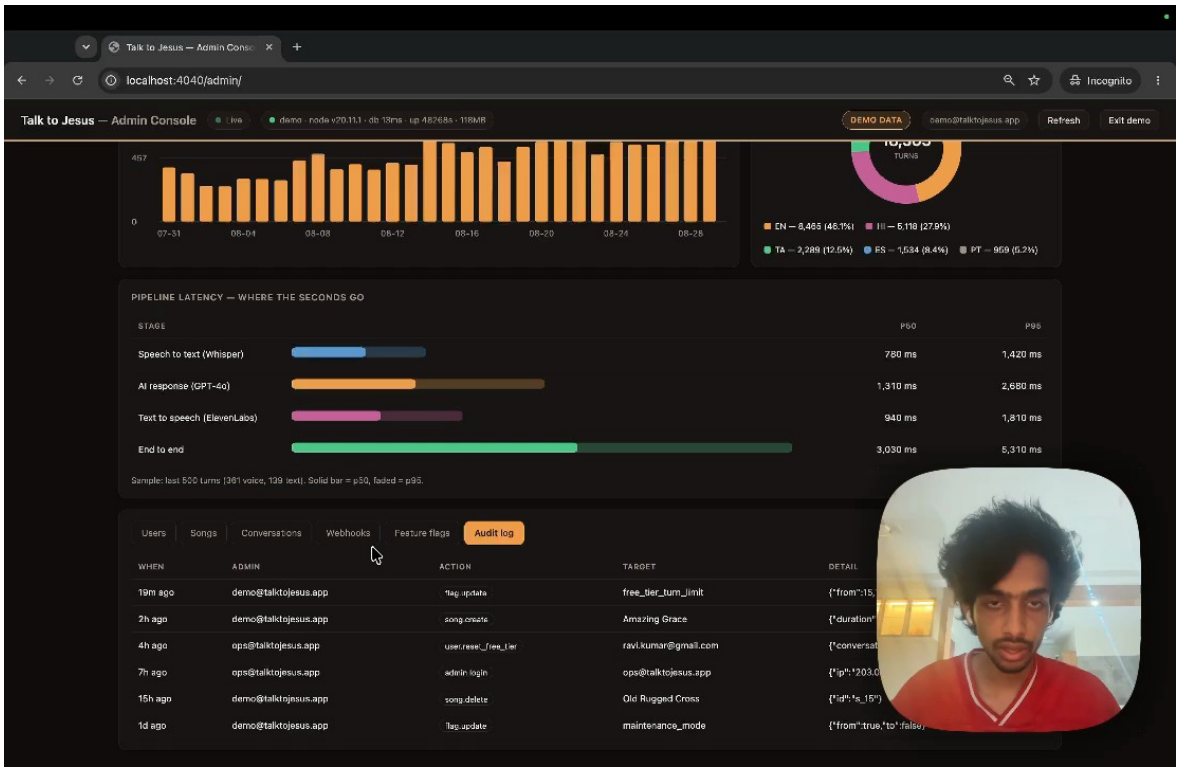

Figure 23 — The administrative audit trail.

4.4 Demonstration video

Two recordings are published, both publicly accessible.

Recording	Duration	Content	Link
Application demonstration	3 min 50 s	Sign-in, bilingual operation, Bible reader, hymn library, a full voice turn, the paywall, Razorpay checkout by UPI, and conversation history	https://drive.google.com/file/d/124faeA_zm2HM7rPP_rWwJPsXSj6iN4QS/view
Administrative console	1 min 13 s	Demonstration mode, then the live instance with the measured latency breakdown	https://drive.google.com/file/d/1UeTyKBB8uDYdNhWjS14s2lbwYxO7xW6p/view

An earlier recording is cited in the repository README and in the Phase 2 and Phase 3 documents. The two links above supersede it.

The administrative recording shows demonstration mode for its first fifty seconds and the live instance thereafter. The distinction is visible on screen — demonstration mode carries a `DEMO DATA` badge — and matters, because the two halves report figures that differ by an order of magnitude for the reasons set out in Section 3.5.

CHAPTER 5: PROJECT EXECUTION EVIDENCE

5.1 Version control evidence

The repository is `github.com/manish-gitx/rn-final-manish`, branch `main`, all commits authored by `manish-gitx`.

Table 15 — Version control activity

Commit	Date	Description	Scale
`d14d3d3`	2026-01-12	Initial commit	2 files
`d903b27`	2026-01-12	Backend and frontend imported into a monorepo	382 files, +26,382
`77f3b6b`	2026-01-12	README and `.gitignore`	+440
`89fac76`	2026-01-12	Problem statement revised	+37 / −13
`988d8c0`	2026-01-12	Multilingual support	9 files, +92 / −37
`ca117d3`	2026-01-13	README updated	+19
`8f20bd9`	2026-02-16	Dockerfile added; subscription defect fixed	11 files
`ce1fbdb`	2026-02-16	`cloudbuild.yaml` added	+38
`fb0d602`	2026-02-16	Dockerfile relocated to the repository root	2 files
`19ffc6c`	2026-02-16	Duplicate Dockerfile removed	−26
`cab8d4d`	2026-02-17	Test suites added	21 files, +7,114
`84c6660`	2026-02-22	Android configuration	1 file
`459acae`	2026-08-05	Administrative console, conversation history, multi-turn context, CI	50 files, +4,703
`ec0ceef`	2026-08-05	Backend test suite made hermetic	+15
`ccfb64b`	2026-08-30	Console	67 files, +1,532 /

Commit	Date	Description	Scale
		authentication, hymn assets, iOS project updates	−1,633

Fifteen commits is a modest history for a project of this size, and the shape is uneven: two large imports, a testing commit, and a substantial feature commit in August that added the console, conversation history, multi-turn context and continuous integration together. A finer-grained history would have made bisecting a regression easier. This is noted as a process observation rather than defended.

5.2 Development timeline

Derived from the commit record and the phase submission dates.

Period	Work
2026-01-05 → 2026-02-02	Phase 1: problem identification, objectives, literature survey, feasibility
2026-01-12 → 2026-01-13	Monorepo established; backend and client imported; multilingual support
2026-02-03 → 2026-02-15	Phase 2: requirements, architecture, technology selection, proof of concept
2026-02-16 → 2026-02-17	Containerisation, Cloud Build pipeline, first test suites
2026-02-16 → 2026-03-03	Phase 3: validation, performance analysis, limitations
2026-02-22	Android configuration
2026-08-05	Administrative console, conversation history, multi-turn context, continuous integration
2026-08-30	Console authentication, hymn assets, iOS project updates
2026-08-30	Capstone documentation, measurement and

Period	Work
	figure capture

CHAPTER 6: CONCLUSION & FUTURE WORK

6.1 Summary of implementation

TalkToJesus delivers what the Study Project set out to build. A user signs in with Google, speaks a question in English or Telugu, and receives a spoken, scripture-citing reply in a pastoral voice. Around that interaction sit a bilingual Bible reader with six translations, a bundled hymn library that works offline, a conversation transcript, and a Razorpay subscription that lifts a three-conversation free tier. Behind it sit twenty-eight API endpoints on Cloud Run, eight PostgreSQL tables, and an administrative console reporting live metrics.

All seven primary objectives are met. Of the four secondary objectives, three are met and one — offline support — is met for the Bible reader and the hymn library but not for the conversation itself, which necessarily requires a network.

6.2 Achievements

The pipeline works in both languages. A Telugu speaker receives a Telugu reply addressed as నా బిడ్డ, and code-switched speech is handled because transcription auto-detects rather than being pinned to the selected language.

The system measures itself. Every turn records the duration of each stage separately. This is the difference between a project that reports that its application works and one that can say where its seconds go — and, as Chapter 3 shows, it is what caught a claim the project had been repeating about itself.

Payments reconcile properly. Subscription state is driven by verified webhooks rather than by trusting the client, signature comparison is timing-safe, and failed verifications are recorded rather than discarded.

Operational levers exist without redeployment. Five runtime flags allow the free tier, maintenance mode, speech synthesis and the context window to be changed live.

The build is gated. Continuous integration compiles and tests both tiers and verifies the container’s contents on every push.

6.3 Limitations

Latency fails its requirement by a factor of five. 27.5 seconds measured against a five-second target. This is the dominant limitation and everything else is secondary to it.

Authorization rests on application code alone. Row-level security is enabled but unpolicied and bypassed by the service key, and no automated test asserts isolation between users.

Secrets are committed. A long-lived JWT, a PostHog key and a Sentry DSN are in source control, and environment backup files sit in the working directory. These must be removed and rotated.

There is no rate limiting and CORS is open. The paid pipeline is protected only by the free-tier counter.

Transcripts are personal data with no policy. They are stored indefinitely with no retention rule, no redaction and no deletion path.

Coverage stops at the service layer. No widget tests, no integration tests, no end-to-end test of the pipeline.

Some controls do nothing. The player's previous and next buttons, and the text-input path that exists in the API with no interface.

Licensing obligations are unmet. Several bundled hymns are CC BY-SA, which requires attribution surfaced to the user; the application has no credits screen.

6.4 Future enhancements

Ordered by the ratio of value to effort, which after Chapter 3 is not the order originally anticipated.

1. Reduce synthesis latency. The single highest-value change. Three steps, in order: first, tighten the persona prompt so replies actually observe the two-to-four sentence target, and re-measure — Section 3.5 argues this alone may account for most of the 19.6 seconds. Second, stream the synthesised audio so playback begins before synthesis completes, which changes perceived latency even if total time is unchanged. Third, evaluate a faster voice model. Re-measure after each step rather than changing all three at once.

2. Test authorization isolation. Add tests asserting that a user cannot read another user's conversations or subscription. Given that the database will not enforce this, it is the most valuable missing test.

3. Remove and rotate the committed secrets. Purge the test token, the analytics key and the error-tracking DSN from source control, rotate all three, and delete the environment backup files before any code submission.

4. Fix the display defects. Strip the emotional tags before rendering (D-01); either implement or remove the player's previous and next controls (D-02).

5. Add an about and credits screen. Required by the CC BY-SA hymn licensing, and the natural home for the third-party attributions in Appendix E.

6. Add rate limiting and restrict CORS. Both are small changes that close the abuse path against a pipeline that costs money per call.

7. Define a data retention policy. Decide how long transcripts persist, give users a delete path, and state the policy in the application.

8. Enlarge the measurement sample. Two turns established an order of magnitude. A few hundred turns would establish a distribution and make the p95 column meaningful.

9. Wire up text input. The API endpoint works and is untested by any interface. Exposing it would give users a fast, silent path — and, because it records `stt_ms = 0`, would make the cost of speech recognition directly comparable in production data.

10. Reconcile the toolchain versions and the deployment configuration. Align Node and Flutter versions between development and CI, and repoint the Cloud Build trigger at the checked-in `cloudbuild.yaml`.

6.5 Concluding remarks

The project set out to close a gap that no existing application addresses in full: conversational, scripture-grounded, regional-language and voice-first at the same time. It does that, and it does it in Telugu with culturally correct address, which was the part most at risk.

The result the project did not anticipate is the more useful one. Building the instrumentation before it was obviously needed meant that when the performance question was finally asked,

the answer already existed in the database — and the answer contradicted what the project had been saying about itself in three separate documents. A system that can be wrong about itself in a way it can detect is a better outcome than one that is quietly wrong. The five-second requirement is not met; the reason is known, it is localised to one stage, and the most probable cause is a prompt rather than a vendor.

REFERENCES

1. Rachakonda, M. (2026). *Study Project Phase 1: Problem Identification and Planning — Talk to Jesus*. BITS Pilani.
2. Rachakonda, M. (2026). *Study Project Phase 2: Requirements, Architecture and Proof of Concept — Talk to Jesus*. BITS Pilani.
3. Rachakonda, M. (2026). *Study Project Phase 3: Implementation, Validation and Results — Talk to Jesus*. BITS Pilani.
4. Kumar, P. *Study Project — Complete Reference Guide, B.Sc. Computer Science*. BITS Pilani.
5. OpenAI. *Speech to text — Whisper API documentation*. https://platform.openai.com/docs/guides/speech-to-text
6. OpenAI. *Chat Completions API documentation*. https://platform.openai.com/docs/guides/text-generation
7. ElevenLabs. *Text to Speech API documentation*. https://elevenlabs.io/docs
8. Google. *Cloud Run documentation*. https://cloud.google.com/run/docs
9. Google. *Google Identity — verifying an ID token*. https://developers.google.com/identity/sign-in/web/backend-auth
10. Razorpay. *Subscriptions API and webhook signature verification*. https://razorpay.com/docs/api/subscriptions/
11. Supabase. *Row Level Security*. https://supabase.com/docs/guides/auth/row-level-security
12. Flutter. *Flutter documentation*. https://docs.flutter.dev
13. Riverpod. *Riverpod documentation*. https://riverpod.dev
14. Jones, M., Bradley, J. and Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF. https://www.rfc-editor.org/rfc/rfc7519

15. Krawczyk, H., Bellare, M. and Canetti, R. (1997). *RFC 2104: HMAC — Keyed-Hashing for Message Authentication*. IETF. https://www.rfc-editor.org/rfc/rfc2104
16. Free Use Bible API. *bible.helloao.org* — the third-party scripture API used by the Bible reader. https://bible.helloao.org
17. YouVersion Bible App. https://www.bible.com — surveyed as an existing system in Phase 1.
18. Pray.com. https://www.pray.com — surveyed as an existing system in Phase 1.

APPENDIX A — USER MANUAL

This appendix is written for someone using the application, not building it. It is also published separately as `07-User-Manual.docx`.

A.1 Installing and starting

The application is not published to the App Store or Google Play. It is installed from a build supplied directly — an `.apk` on Android, or through TestFlight or a development build on iOS. On first launch it requests no permissions; the microphone is requested later, at the moment it is first needed.

A.2 Signing in

The login screen shows a single control, **Sign up with Google** (Figure 1). Tap it, choose a Google account, and accept the consent prompt (Figure 2). The application creates an account on first sign-in and takes you to the home screen. There is no password to set or remember.

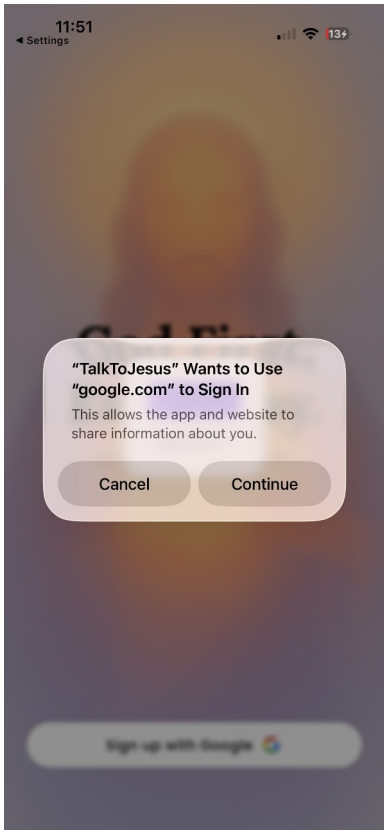

Figure 2 — The Google consent prompt shown by iOS on first sign-in.

Demonstration account. Tapping the headline "God First, Every Day." five times reveals an additional **Enter with Demo Account** button. This exists for demonstrations and does not require a Google account.

A.3 The home screen

The home screen (Figure 3) has four elements.

- **The menu button**, top left, opens your profile drawer.
- **The language toggle**, top centre, switches between **EN** and **ಕ**.
- **The status badge**, top right, shows subscription state: a green dot when a subscription is active, a crossed icon when it is not. Tapping the crossed icon opens the subscription options.
- **The voice bar**, along the bottom, is the main control. Above it sit two buttons, **Jesus Songs** and **Bible**.

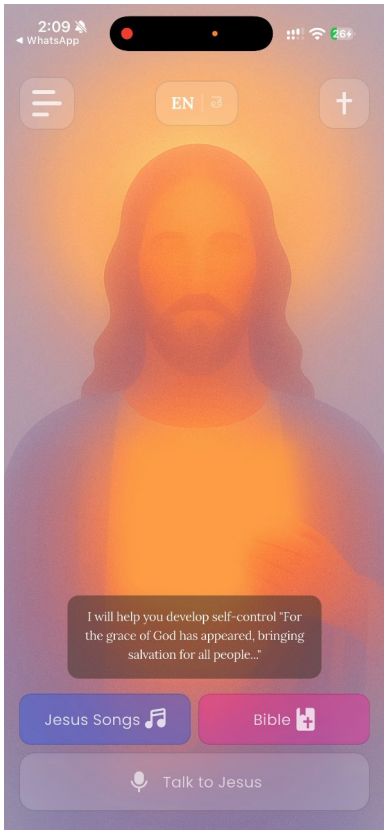

Figure 3 — The home screen in English.

A.4 Having a conversation

1. Tap the voice bar labelled **Talk to Jesus**.
2. The first time, iOS or Android will ask for microphone permission. Allow it. If you previously denied it, the application offers to open system settings.
3. Recording begins (Figure 6). Speak naturally. There is no time limit, but keep it to a minute or so.
4. Tap the square **stop** button when finished, or the × to cancel without sending.
5. The bar shows *Listening to your heart* while the reply is prepared.
6. The reply appears as text and plays aloud automatically.

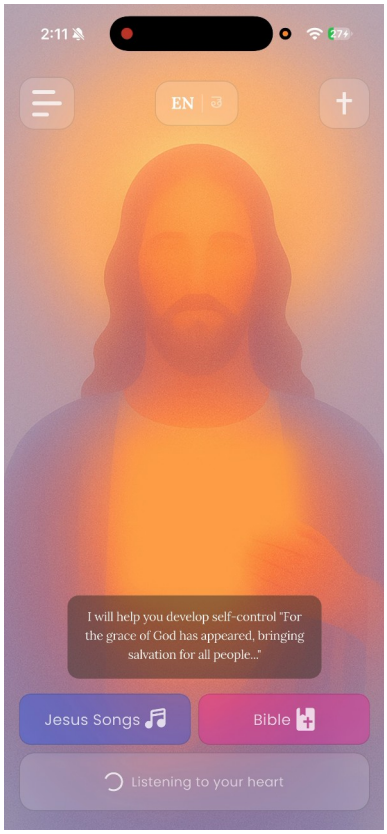

Figure 7 — While the reply is being prepared, the bar reads "Listening to your heart".

Expect to wait. A reply currently takes around 25 to 30 seconds. This is longer than intended; Section 3.5 of the main report explains why. The application has not frozen.

Switching language. Tap **EN | ತ**. Every visible label changes immediately and the reply you receive next will be in the selected language (Figure 5). You do not need to restart.

A note on what you will see. Replies currently include bracketed words such as `[gently]` or `[prayerfully]`. These are instructions to the voice engine that were not meant to be displayed. They are a known defect (D-01) and do not form part of the message.

A.5 Conversation history

Open the menu, top left, and choose **Conversation History** (Figure 8). Each card shows the language, how long ago the exchange happened, what you said and what was said in reply. Pull down to refresh. History is private to your account.

A.6 Reading the Bible

Tap **Bible** on the home screen.

- **Change book or chapter** — tap the book name in the header, search or scroll, then pick a chapter from the grid (Figure 11).
- **Change translation** — tap the translation code in the header. Six are offered, including two in Telugu (Figure 10).
- Your position is remembered per book and translation, so returning takes you back where you were.
- Chapters you have already read remain available without a network.

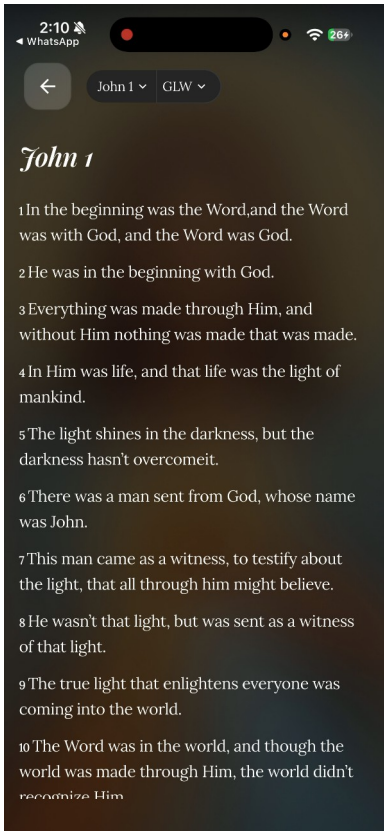

Figure 12 — The Bible reader in English, showing John 1.

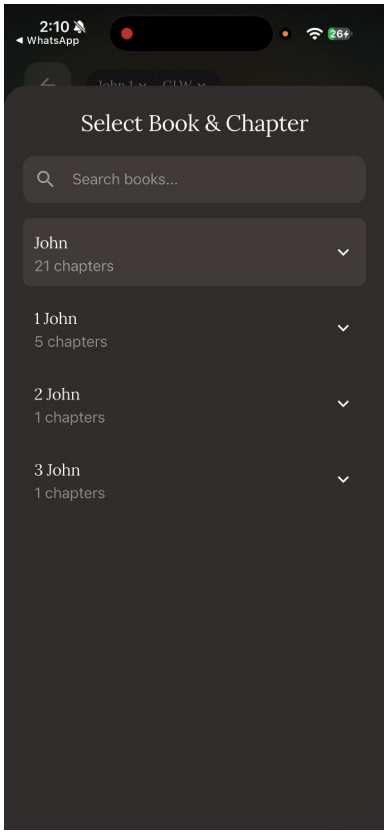

Figure 11 — The book and chapter selector, with search.

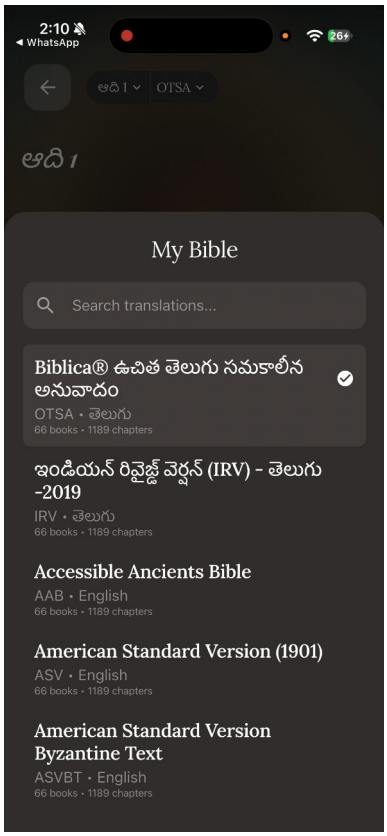

Figure 10 — The translation selector. Six translations are offered, two of them Telugu.

A.7 Listening to hymns

Tap **Jesus Songs** (Figure 13). Twelve hymns are bundled with the application and play without a network. Tap one to open the player (Figure 14), which offers play, pause and a seek bar.

Known limitation. The previous and next buttons in the player do not currently work (defect D-02). Go back to the list to change track.

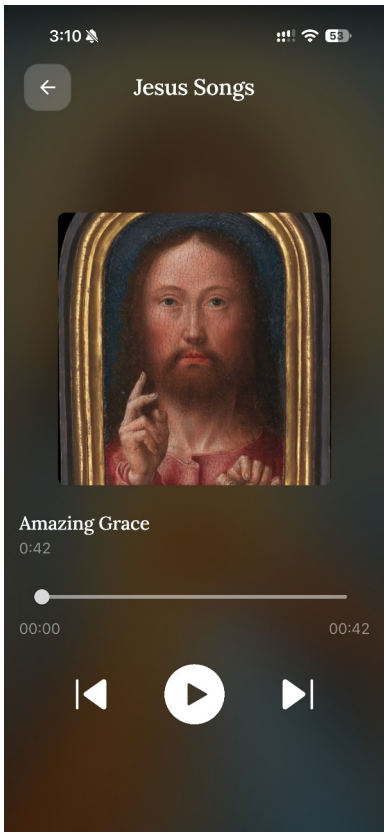

Figure 14 — The hymn player.

A.8 Subscribing

The first three conversations are free. After that the subscription sheet appears (Figure 15).

1. The plan is ₹**499 per month** for twelve monthly cycles.
2. Tap it to open Razorpay checkout (Figure 16).
3. Enter a mobile number and email, then choose UPI, card or e-mandate.
4. Complete the payment in your UPI application if prompted.
5. On success you return to the application and the status badge turns green (Figures 17 and 18).

Payments are handled entirely by Razorpay. The application never sees your card or UPI credentials.

A.9 Signing out

Open the menu and choose **Sign Out**. This clears the stored token from the device. Your conversation history is retained on the server and reappears when you sign back in.

A.10 Troubleshooting

Symptom	Likely cause	What to do
The voice bar does nothing	Microphone permission denied	Open system settings and enable the microphone for TalkToJesus
Reply takes 30 seconds	Expected behaviour today	Wait. See Section 3.5 of the main report
Reply arrives as text with no audio	Speech synthesis unavailable or disabled	The text reply is still correct. Try again later
“Subscription required” after three conversations	Free tier exhausted	Subscribe, or wait for an administrator to reset the counter
Payment succeeded but the badge is still inactive	Webhook not yet delivered	Reopen the application after a few seconds; a 24-hour grace period covers this window
Bible will not load	No network and the chapter is not cached	Connect to a network once; the chapter is then cached
Signed out unexpectedly	Token expired or was rejected	Sign in again
Hymns will not play	Audio routed elsewhere or device muted	Check the volume and any connected Bluetooth device

A.11 Frequently asked questions

Is this a real person? No. Replies are generated by a language model under a persona prompt. It is not a substitute for a pastor, a counsellor or a doctor.

Should I rely on it in a crisis? No. The persona is instructed to defer to human pastoral care for matters of crisis. Please contact a person you trust or an appropriate professional service.

Are the scripture citations reliable? They are generated by the model rather than retrieved from a verified corpus, and their accuracy was not formally verified (Section 3.8). Check any citation that matters to you.

Who can see my conversations? They are stored on the project's server and are visible to an administrator through the console. There is currently no retention policy or deletion path — this is a recorded limitation.

Does it work offline? The Bible reader and the hymn library do, once cached. Conversation requires a network.

Can I type instead of speaking? Not in this build. The capability exists in the API but has no interface.

APPENDIX B — INSTALLATION GUIDE

Also published separately as `08-Installation-Guide.docx`.

B.1 Prerequisites

Requirement	Version	Notes
Node.js	20 LTS	Production runs Node 20; development has been run on Node 24.7
npm	10+	Ships with Node 20
Flutter SDK	3.38.6 stable	Continuous integration pins this; 3.32.7 has been used locally
Xcode	15+	iOS builds only
Android Studio	Recent	Android builds only

Requirement	Version	Notes
Docker	Recent	Container builds only
`gcloud` CLI	Recent	Cloud Run deployment only

Accounts required: Supabase, OpenAI, ElevenLabs, Razorpay, Google Cloud with an OAuth client, and Firebase.

B.2 Getting the source

```
git clone https://github.com/manish-gitx/rn-final-manish.git
cd rn-final-manish
```

The repository is a monorepo: `TalkToJesus-backend/` is the API, `talktojesus-frontend/` is the Flutter application, and the `Dockerfile` and `cloudbuild.yaml` at the root build and deploy the backend.

B.3 Provisioning the database

In the Supabase SQL editor, run these three files **in order**:

1. `TalkToJesus-backend/supabase-setup.sql` — creates `users`, `songs`, `plans`, `subscriptions`
2. `TalkToJesus-backend/supabase-admin-setup.sql` — adds `is_admin`, `conversation_logs`, `webhook_events`, `feature_flags`, `admin_audit_log`
3. `TalkToJesus-backend/admin-bootstrap.sql` — idempotent; ends with a verification query

Running them out of order fails, because the second alters a table the first creates.

B.4 Backend configuration

Create `TalkToJesus-backend/.env`. These are the variable names the code actually reads; note that two of them differ from the names given in the repository README, and the code is authoritative.

Variable	Required	Notes
`PORT`	No	Defaults to 4040; the

Variable	Required	Notes
		container sets 8080
`NODE_ENV`	No	Selects Razorpay credentials and the plan filter
`LOG_LEVEL`	No	Defaults to `info`
`SUPABASE_URL`	**Yes**	Module throws at import if absent
`SUPABASE_KEY`	**Yes**	Service-tier key. **The README calls this `SUPABASE_SERVICE_ROLE_KEY`; the code reads `SUPABASE_KEY`**
`JWT_SECRET`	**Yes**	Module throws at import if absent
`GOOGLE_CLIENT_ID_WEB`	Yes (≥1 of 3)	**The README shows a single `GOOGLE_CLIENT_ID`; the code reads three separate variables**
`GOOGLE_CLIENT_ID_IOS`	Yes (≥1 of 3)	
`GOOGLE_CLIENT_ID_ANDROID`	Yes (≥1 of 3)	
`RAZORPAY_KEY_ID_DEV` / `_SECRET_DEV` / `_WEBHOOK_SECRET_DEV`	**Yes** when not production	Module throws at import if absent
`RAZORPAY_KEY_ID_PROD` / `_SECRET_PROD` / `_WEBHOOK_SECRET_PROD`	**Yes** in production	
`OPENAI_API_KEY`	**Yes**	Used for both transcription and generation
`OPENAI_MODEL`	No	Defaults to `gpt-4o`

Variable	Required	Notes
`OPENAI_MAX_TOKENS`	No	Defaults to 800
`OPENAI_TEMPERATURE`	No	Defaults to 0.7
`ELEVENLABS_API_KEY`	No	If absent, replies are text-only rather than failing
`ELEVENLABS_VOICE_ID`	Yes for speech	
`ELEVENLABS_MODEL`	No	Defaults to `eleven_multilingual_v2`
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	Yes for the console	Console login returns 500 if unset

There is no `.env.example` in the repository; this table serves that purpose.

A note on ports. Three different ports appear across the project's documentation — 3000, 4040 and 5000. The code defaults to **4040** and the container listens on **8080**. Use those.

B.5 Running the backend

```
cd TalkToJesus-backend
npm ci
npm run dev          # nodemon on port 4040
```

Verify with `curl http://localhost:4040/`, which returns a JSON health banner. The administrative console is then at `http://localhost:4040/admin`.

Other commands:

```
npm run build        # tsc, also the CI typecheck gate
npm start            # node dist/index.js
npm test             # 66 tests, no credentials required
npm run seed:demo    # ~200 synthetic conversations across 30 days
npm run seed:demo:clear
```

`npm run seed:demo` writes clearly-marked demonstration rows: addresses end `@demo.talktojesus.local` and seeded replies carry a marker. It is demonstration data, not traffic.

B.6 Client configuration

Three files are required and are deliberately not in source control:

- `talktojesus-frontend/android/app/google-services.json`
- `talktojesus-frontend/ios/Runner/GoogleService-Info.plist`
- `talktojesus-frontend/lib/firebase_options.dart` — generated by `flutterfire configure`

For a release Android build you also need `android/key.properties` and a keystore; without them the build falls back to unsigned.

B.7 Running the client

```
cd talktojesus-frontend
flutter pub get
flutter run                                                    #
against the deployed backend
flutter run --dart-define=API_BASE_URL=http://localhost:4040   #
against a local backend
```

`API_BASE_URL` is the only compile-time define that takes effect. The others named in `EnvironmentConfig` are inert, because `main.dart` initialises the flavour with literal values instead of reading them.

B.8 Building for release

```
flutter build apk --release
flutter build appbundle --release
flutter build ios --release
```

Before submitting to Google Play, change the application identifier from the scaffold default `com.example.talktojesus` (defect D-09).

B.9 Container build and deployment

From the repository root — the Dockerfile expects root as its build context:

```
docker build -t talktojesus-backend .
docker run -p 8080:8080 --env-file TalkToJesus-backend/.env
talktojesus-backend
```

Deployment is Cloud Build to Artifact Registry to Cloud Run, service `talktojesus-backend` in `us-central1`. Note the caveat in Section 4.2.2: the live trigger currently carries its configuration inline and ignores the checked-in `cloudbuild.yaml`.

B.10 Administrative access

The console at `/admin` accepts the `ADMIN_EMAIL` and `ADMIN_PASSWORD` pair and issues a token carrying a console-administrator claim. That principal is synthetic — it has no row in `users`. The consequence is that there is no `is_admin` flag to clear, so **rotating the password is the only revocation, and tokens already issued remain valid until they expire.**

To grant in-application administrative access to a real user instead, set `is_admin = true` on their `users` row.

B.11 Troubleshooting

Symptom	Cause	Resolution
Backend exits at startup	A required variable is missing	Check `SUPABASE_URL`, `SUPABASE_KEY`, `JWT_SECRET` and the Razorpay set
503 with `SCHEMA_NOT_PROVISIONED`	Admin migration not run	Run `supabase-admin-setup.sql`
Every login fails	No Google client ID configured	Set at least one of the three
`/admin` returns 404 as an admin	The admin gate returns 404 by design	Confirm `is_admin` is exactly `true`, or use the console credentials
Console login returns 500	`ADMIN_EMAIL` / `ADMIN_PASSWORD` unset	Set both and restart
Replies have no audio	ElevenLabs key missing, quota exhausted, or `tts_enabled` false	Check the key and the flag

Symptom	Cause	Resolution
Client cannot reach a local backend	Device cannot resolve `localhost`	Use the host's LAN address in `API_BASE_URL`
Payment succeeds, app still locked	Webhook not delivered	Check the webhook secret and the console's webhook tab

APPENDIX C — SOURCE CODE

Repository: **https://github.com/manish-gitx/rn-final-manish**

Branch `main`. The submitted revision is `ccfb64b`.

Path	Contents
`TalkToJesus-backend/src/api/routes/`	28 endpoint definitions
`TalkToJesus-backend/src/api/controllers/`	Request handling, validation, response shaping
`TalkToJesus-backend/src/api/services/`	Business logic and all external calls
`TalkToJesus-backend/src/api/middlewares/`	Authentication and the administrative gate
`TalkToJesus-backend/src/config/prompts.ts`	The persona prompt, parameterised by language
`TalkToJesus-backend/src/__tests__/`	66 Jest cases across 10 suites
`TalkToJesus-backend/public/admin/index.html`	The administrative console, one file
`TalkToJesus-backend/*.sql`	Schema provisioning, in the order given in Appendix B.3
`talktojesus-frontend/lib/presentation/pages/`	The eight screens
`talktojesus-frontend/lib/data/`	API client, authentication, token storage, conversation, payment

Path	Contents
`services/`	
`talktojesus-frontend/test/`	75 Flutter cases across 9 files
`docs/figures/`	The 29 report figures with an index
`docs/evidence/`	Captured test output and the extracted case list
`scripts/docs/`	The scripts that generate the figures and the test tables
`Dockerfile`, `cloudbuild.yaml`, `.github/workflows/ci.yml`	Build, deployment and continuous integration

APPENDIX D — DEMONSTRATION VIDEO

Recording	Duration	Link
Application demonstration	3 min 50 s	https://drive.google.com/file/d/124faeA_zm2HM7rPP_rWwJPsXSj6iN4QS/view
Administrative console	1 min 13 s	https://drive.google.com/file/d/1UeTyKBB8uDYdNhWjS14s2lbwYxO7xW6p/view

Both links are publicly readable. The figures in this report were sampled from these recordings; `docs/figures/FIGURES.md` maps each figure to its source and timestamp.

APPENDIX E — THIRD-PARTY COMPONENTS AND LICENSING

This appendix supports the plagiarism and originality declaration, which is published separately as `06-Plagiarism-Compliance.docx`.

E.1 What is original

All application source code in `TalkToJesus-backend/src/`, `talktojesus-frontend/lib/` and `talktojesus-frontend/test/`, the database schema, the persona

prompt, the administrative console, the continuous integration and deployment configuration, and this report were written by the candidate. No code was copied from another project.

E.2 Backend dependencies

Licences below were read from each package's own metadata in `node_modules`, not assumed.

Package	Licence	Package	Licence
`@supabase/supabase-js`	MIT	`morgan`	MIT
`axios`	MIT	`multer`	MIT
`cors`	MIT	`razorpay`	MIT
`dotenv`	BSD-2-Clause	`winston`	MIT
`express`	MIT	`zod`	MIT
`form-data`	MIT	`jest`	MIT
`google-auth-library`	Apache-2.0	`ts-jest`	MIT
`jsonwebtoken`	MIT	`typescript`	Apache-2.0
		`nodemon`, `ts-node`, `ts-node-dev`, all `@types/*`	MIT

E.3 Client dependencies

Licences below were read from each package's `LICENSE` file in the pub cache.

Package	Licence	Package	Licence
`flutter_riverpod`	MIT	`google_sign_in`	BSD-3-Clause
`flutter_svg`	MIT	`http`	BSD-3-Clause
`audioplayers`	MIT	`path`, `path_provider`	BSD-3-Clause
`permission_hand`	MIT	`shared_preferen`	BSD-3-Clause

Package	Licence	Package	Licence
`ler`		`ces`	
`razorpay_flutte r`	MIT	`sqflite`	BSD-3-Clause
`posthog_flutter`	MIT	`record`	BSD-3-Clause
`sentry_flutter`	MIT	`connectivity_pl us`	BSD-3-Clause
`in_app_review`	MIT	`crypto`	BSD-3-Clause
`lottie`	MIT	`google_fonts`	BSD-3-Clause
`cupertino_icons`	MIT	`shimmer`	BSD-3-Clause
`flutter_launche r_icons`	MIT	`url_launcher`	BSD-3-Clause
		`firebase_core`, `firebase_auth`, `cloud_firestore`	BSD-3-Clause

Typefaces Poppins and Lora are fetched at runtime by `google_fonts` and are licensed under the SIL Open Font License.

E.4 External services

Service	Role	Terms
OpenAI (Whisper, GPT-4o)	Transcription and generation	Commercial API, per-use
ElevenLabs	Speech synthesis	Commercial API, per-use
Google Identity	Authentication	Google API terms
Razorpay	Payments	Commercial agreement
Supabase	Managed PostgreSQL	Commercial
Google Cloud Run	Hosting	Commercial
bible.helloao.org	**Scripture text for the Bible reader**	**Free public API — third-party, not operated by this project**

The Bible reader's content comes entirely from a third-party public API. No scripture text is authored or hosted by this project.

E.5 Bundled media

Twelve hymn excerpts, approximately forty-two seconds each at 64 kbps mono, sourced from Wikimedia Commons and the Internet Archive. Each was trimmed, given a fade, and loudness-normalised; no other alteration was made. Nine are public domain. **Three are CC BY-SA and carry a continuing obligation:**

Track	Licence
What a Friend We Have in Jesus	CC BY-SA 4.0
Holy, Holy, Holy	CC BY-SA 4.0
Abide With Me	CC BY-SA 3.0

CC BY-SA requires attribution and share-alike on redistribution. The full record, with source links per track, is kept alongside the assets at `talktojesus-frontend/assets/music/hymns/ATTRIBUTION.md`, which itself states that the credits should be surfaced in an about screen.

The application has no about screen. This obligation is therefore currently unmet, and is recorded as defect D-08 with the remedy listed in Section 6.4.

Album artwork is twelve public-domain paintings from the Metropolitan Museum of Art Open Access collection (CC0), centre-cropped and downscaled. Artists include Gerard David, Hans Memling, Eugène Delacroix, Petrus Christus, Ambrosius Benson, Sebastiano Ricci and Joos van Wassenhove; the per-track record is in the same attribution file.

E.6 Prior work by the candidate

This report restates material from the candidate's own Phase 1, Phase 2 and Phase 3 submissions — the abstract, the objectives, the literature survey and the requirement definitions in particular. Those documents are cited as references 1 to 3. Section 1.6 and Section 3.6 record every point at which this report corrects or supersedes them.

E.7 Similarity check

To be completed by the candidate. The institutional similarity check has not been run from within this project. Reported similarity is expected to be concentrated in the restated requirement definitions and objectives, which are the candidate's own prior submissions and are cited as such.

Similarity score: _____ Tool: _____ Date: _____